Министерство образования и науки Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Курсовая работа по дисциплине

«Функциональное программирование»
«Рекомендательная система акустической пригодности помещения по его назначению»

Выполнила студентка группы 5130201/40003 _____ Рузина В.А.

Проверил, к.т.н. _____ Моторин Д.Е.

« ___ » _____ 2026 г.

г.Санкт-Петербург, 2026 г.

Содержание

Введение **4**

1 Предметная область и математический аппарат **6**

1.1 Акустика помещений и время реверберации как критерий пригодности 6

1.2 Геометрия помещения 6

1.3 Эквивалентная площадь звукопоглощения 7

1.4 Время реверберации: формулы Сэбина и Эйринга 8

1.5 Сравнение с нормой и оценка пригодности 9

1.6 Алгоритм формирования рекомендаций 9

2 Исходные данные и их обработка **13**

2.1 Источник и формат входных данных 13

2.2 Модель предметной области: типы данных 14

2.3 Хранение и загрузка помещений в формате JSON 19

3 Описание разработки проекта **21**

3.1 Архитектура программного комплекса 21

3.1.1 Модуль Types 22

3.1.2 Модуль Input.CSV 22

3.1.3 Модуль Calculation.Geometry 22

3.1.4 Модуль Calculation.Absorption 24

3.1.5 Модуль Calculation.Reverberation 25

3.1.6 Модуль Evaluation.Suitability 27

3.1.7 Модуль Recommendation.Acoustic 28

3.1.8 Модуль Output.Report 29

3.1.9 Модуль Storage.JSON 29

3.1.10 Модуль Batch.Check 30

3.1.11 Модуль Input.CLI 31

3.1.12 Модуль Log.Logger 32

3.2 Технические особенности реализации 32

3.2.1 Единицы измерения и представление данных 32

3.2.2 Обработка ошибок через Either 33

3.2.3 Логирование действий и событий программы 33

3.3 Тестирование 33

4 Примеры работы программы 35

4.1 Создание помещения и проверка на пригодность 35
4.2 Создание и запись в файл 38
4.3 Пакетная проверка из файла 39
4.4 Просмотр сохранённых помещений 40
4.5 Удаление помещения 41
4.6 Обработка ошибок 41

5 Руководство по эксплуатации 42

5.1 Требования к окружению 42
5.2 Запуск программы 43
5.3 Создание и проверка помещения 44
5.3.1 Ввод названия и размеров 44
5.3.2 Выбор материалов поверхностей 45
5.3.3 Ввод окон, дверей и элементов декора 46
5.3.4 Ввод сидячих мест 47
5.3.5 Проверка помещения и формирование рекомендаций 47
5.4 Создание и сохранение помещения 49
5.5 Просмотр сохранённых помещений 50
5.6 Удаление помещения 50
5.7 Пакетная проверка помещений 51
5.8 Обработка ошибок и журналирование 52
5.9 Завершение работы 53

Заключение 54

История коммитов проекта 56

Список литературы 58

Введение

При организации мероприятий — концертов, конференций, лекций — качество восприятия речи и музыки в помещении определяется не только его размерами, но и акустическими свойствами отделки и обстановки. Одно и то же помещение может отлично подходить для тихой лекции и быть совершенно непригодным для живого концерта: избыточно «гулкое» помещение делает речь неразборчивой, а чрезмерно «заглушённое» — лишает музыку живости. Ручная оценка акустической пригодности требует привлечения инженера-акустика и проведения измерений, что не всегда доступно на этапе выбора или подготовки площадки.

Ключевой количественной характеристикой, определяющей субъективное восприятие звука в помещении, является **время реверберации** T_{60} — время, за которое уровень звука после выключения источника снижается на 60 дБ. Для каждого **назначения помещения** — студии звукозаписи, переговорной комнаты, учебной аудитории, конференц-зала, кинотеатра, концертного зала, ресторана, спортзала и других — существует рекомендуемый диапазон значений T_{60}, вне которого помещение считается непригодным без дополнительной акустической коррекции. Классическим инструментом инженерной оценки служит формула Сэбина, связывающая время реверберации с объёмом помещения и суммарным звукопоглощением его поверхностей и предметов обстановки. При больших значениях среднего коэффициента звукопоглощения применяется формула Эйринга, дающая более точный результат.

В настоящей работе разработан **консольный программный комплекс на языке Haskell**, который по геометрическим и материаловедческим характеристикам помещения:

- рассчитывает время реверберации по формуле Сэбина или Эйринга **на трёх октавных частотах** (125, 500 и 2000 Гц);
- сравнивает полученные значения с нормативным диапазоном **выбранного назначения помещения**;
- выносит заключение о пригодности помещения;

- в случае несоответствия формирует **пошаговые рекомендации** по добавлению или удалению звукопоглощающих элементов интерьера (штор, ковров, акустических панелей) для достижения требуемого диапазона;

- поддерживает создание, сохранение, просмотр, удаление и **пакетную проверку** помещений;

- сохраняет результат работы в текстовый отчёт и ведёт журнал действий пользователя и событий программы.

Программа реализована с использованием системы сборки **Stack**, библиотек **cassava** (чтение CSV), **aeson** (работа с JSON), **QuickCheck** (тестирование вычислительных функций). Архитектура приложения модульная: типы данных предметной области отделены от вычислительной логики, модули оценки пригодности и формирования рекомендаций используют результаты расчётов через типобезопасные интерфейсы. Обработка ошибок выполняется через тип `Either AppError`, что исключает аварийное завершение программы при некорректном пользовательском вводе или повреждённых файлах данных.

Рассматривается полный цикл разработки — от математической постановки задачи до экспериментальной проверки на модельных примерах помещений.

1 Предметная область и математический аппарат

1.1 Акустика помещений и время реверберации как критерий пригодности

Помещение моделируется как замкнутый объём V (м^3), ограниченный набором поверхностей $\{(S_i, \alpha_{i,f})\}$, где S_i — площадь i-й поверхности (пол, потолок, стены, окна, двери) в м^2, а $\alpha_{i,f} \in [0, 1]$ — коэффициент звукопоглощения материала этой поверхности на частоте f.

Помимо стационарных поверхностей, помещение содержит набор регулируемых элементов интерьера $\{(a_j, \alpha_{j,f})\}$ — штор, ковров, мягкой мебели, акустических панелей. Каждый элемент характеризуется площадью a_j (м^2) и коэффициентом поглощения $\alpha_{j,f}$ на частоте f.

Каждому **типу назначения** E (студия звукозаписи, переговорная комната, учебная аудитория, конференц-зал, кинотеатр, концертный зал, ресторан, спортзал и т. д.) ставится в соответствие рекомендуемый диапазон времени реверберации $[T_{E,\min,f}, T_{E,\max,f}]$ на каждой частоте f. Чем выше требования к разборчивости речи, тем короче должно быть допустимое время реверберации; для музыкальных назначений, напротив, требуется более «живое» звучание с большим временем реверберации.

1.2 Геометрия помещения

Объём помещения:

$$V = l \cdot w \cdot h, \tag{1}$$

где l — длина, w — ширина, h — высота помещения (м).

Площади поверхностей (пол, потолок и четыре стены):

$$S_{\text{пола}} = S_{\text{потолка}} = lw, \quad S_{\text{стены 1,3}} = lh, \quad S_{\text{стены 2,4}} = wh, \tag{2}$$

$$S_{\text{пов}} = 2(lw + lh + wh). \tag{3}$$

Площадь зоны сидячих мест на полу:

$$S_{\text{мест}} = \sum_j n_j s_j, \tag{4}$$

где n_j — количество мест j-го типа, s_j — площадь пола, занимаемая одним местом (м^2).

Общая площадь поверхностей без площади, занятой сидячими местами:

$$S_{\text{общ}} = S_{\text{пов}} - S_{\text{мест}}. \tag{5}$$

Свободная площадь поверхности i:

$$S_{\text{своб},i} = S_i - \sum_k S_{ik} - \begin{cases} S_{\text{мест}}, & i = \text{пол}, \\ 0, & i \neq \text{пол}, \end{cases} \tag{6}$$

где S_{ik} — площадь k-го участка (элемента интерьера) на поверхности i (м^2).

1.3 Эквивалентная площадь звукопоглощения

Для каждой частоты f звукопоглощение поверхностей (зона мест исключена, её поглощение входит в $A_{\text{мебель},f}$):

$$A_{\text{пов},f} = \sum_i \left[\alpha_{i,f} \cdot S_{\text{своб},i} + \sum_k \alpha_{ik,f} \cdot S_{ik} \right], \tag{7}$$

где $\alpha_{i,f}$ — коэффициент звукопоглощения базового материала поверхности i на частоте f; $\alpha_{ik,f}$ — коэффициент звукопоглощения элемента интерьера на участке k поверхности i.

Звукопоглощение мебели и людей:

$$A_{\text{мебель},f} = \sum_j A_{j,f} \cdot n_j, \tag{8}$$

где $A_{j,f}$ — эквивалентная площадь звукопоглощения единицы мебели j-го типа на частоте f (м^2·сэбин), задаётся в каталоге.

Добавочное звукопоглощение (светильники, вентиляционные решётки, неровности поверхностей):

$$A_{\text{доп},f} = \alpha_{\text{доп},f} \cdot S_{\text{общ}}. \tag{9}$$

Значения $\alpha_{\text{доп},f}$ для помещения простой формы берутся как 50 % от середины нормативного диапазона:

f, Гц	125	500	2000
Диапазон $\alpha_{\text{доп}}$	0,08–0,09	0,04–0,05	0,04–0,05
$\alpha_{\text{доп},f}$	0,0425	0,0225	0,0225

Суммарная эквивалентная площадь звукопоглощения (сэбин; 1 сэбин = 1 м^2 полностью поглощающей поверхности):

$$A_f = A_{\text{пов},f} + A_{\text{мебель},f} + A_{\text{доп},f}. \tag{10}$$

Средний коэффициент звукопоглощения и его функция:

$$\alpha_{\text{ср},f} = \frac{A_f}{S_{\text{общ}}}, \quad \varphi(\alpha_{\text{ср},f}) = -\ln(1 - \alpha_{\text{ср},f}). \tag{11}$$

1.4 Время реверберации: формулы Сэбина и Эйринга

Время реверберации на частоте f определяется по формуле, выбираемой по значению $\alpha_{\text{ср},f}$:

$$T_f = \begin{cases} \dfrac{0{,}163 \cdot V}{\alpha_{\text{ср},f} \cdot S_{\text{общ}}} = \dfrac{0{,}163 \cdot V}{A_f}, & \alpha_{\text{ср},f} \le 0{,}2 \quad \text{(Сэбин)}, \\ \dfrac{0{,}163 \cdot V}{-S_{\text{общ}} \ln(1 - \alpha_{\text{ср},f})} = \dfrac{0{,}163 \cdot V}{S_{\text{общ}} \varphi(\alpha_{\text{ср},f})}, & \alpha_{\text{ср},f} > 0{,}2 \quad \text{(Эйринг)}, \end{cases} \tag{12}$$

где T_f — время реверберации (с); V — объём (м^3); 0,163 — константа (с/м). Знак «минус» в знаменателе формулы Эйринга компенсирует отрицательное значение $\ln(1-\alpha_{\text{ср},f})$, поэтому знаменатель положителен. Расчёт выполняется при $A_f > 0$, $S_{\text{общ}} > 0$ и $\alpha_{\text{ср},f} < 1$. Выбор формулы выполняется **отдельно для каждой частоты**. Затухание звука в воздухе не учитывается.

1.5 Сравнение с нормой и оценка пригодности

Минимальное и максимальное время реверберации $T_{\min,f}$, $T_{\max,f}$ определяется по справочнику нормативов для выбранного назначения помещения.

Отклонение на частоте f:

$$\delta_f = \begin{cases} T_{\min,f} - T_f, & T_f < T_{\min,f}, \\ T_f - T_{\max,f}, & T_f > T_{\max,f}, \\ 0, & T_{\min,f} \le T_f \le T_{\max,f}. \end{cases} \tag{13}$$

Относительное отклонение помещения:

$$r = \max_f \frac{\delta_f}{T_{\text{пр},f}}, \tag{14}$$

где $T_{\text{пр},f}$ — нарушенная граница ($T_{\min,f}$ или $T_{\max,f}$); при $\delta_f = 0$ слагаемое принимается равным нулю.

Шкала оценки (проектное решение):

Условие	Оценка
$r = 0$ (все три частоты в норме)	пригодно
$0 < r \le 0{,}1$	условно пригодно
$r > 0{,}1$	непригодно

Результаты сравнения выводятся таблицей по трём частотам.

1.6 Алгоритм формирования рекомендаций

Поскольку перебор всех подмножеств регулируемых элементов интерьера экспоненциален, применяется **жадный пошаговый алгоритм**. Сначала вычисляется прирост эквивалентной площади поглощения от размещения элемента, а затем подбирается пара «элемент — поверхность» с наибольшим приростом на худшей частоте.

Прирост поглощения и прогноз. Элемент интерьера с коэффициентом $\alpha_{e,f}$ и площадью S_3, размещаемый на поверхности с коэффициентом $\alpha_{p,f}$, закрывает часть поверхности. Изменение эквивалентной площади поглощения на частоте f:

$$\Delta A_f = d_f \cdot S_3, \quad d_f = \begin{cases} \alpha_{e,f} - \alpha_{p,f}, & \text{добавление элемента,} \\ -(\alpha_{e,f} - \alpha_{p,f}), & \text{удаление элемента.} \end{cases} \tag{15}$$

После изменения вычисляются $A'_f = A_f + \Delta A_f$, затем $\alpha'_{\text{ср},f}$ и T'_f с повторным выбором формулы.

Обратная задача: поглощение, соответствующее заданному T. Пусть $X = 0{,}163 \cdot V/T$; тогда

$$A_{\text{цель}}(T) = \begin{cases} X, & X/S_{\text{общ}} \le 0{,}2 \quad \text{(Сэбин)}, \\ S_{\text{общ}}\left(1 - e^{-X/S_{\text{общ}}}\right), & X/S_{\text{общ}} > 0{,}2 \quad \text{(Эйринг)}. \end{cases} \tag{16}$$

Вторая ветвь следует из $\varphi(\alpha_{\text{ср}}) = 0{,}163 \cdot V/(T \cdot S_{\text{общ}})$ и $A = S_{\text{общ}}(1 - e^{-\varphi})$. Функция $A_{\text{цель}}$ убывает с ростом T, поэтому границам нормы соответствуют границы по поглощению:

$$A_{\min,f} = A_{\text{цель}}(T_{\max,f}), \quad A_{\max,f} = A_{\text{цель}}(T_{\min,f}). \tag{17}$$

Помещение на частоте f в норме, когда $A_{\min,f} \le A_f \le A_{\max,f}$.

Недостающее (избыточное) поглощение.

$$\Delta A_{\text{треб},f} = \begin{cases} A_{\min,f} - A_f > 0, & T_f > T_{\max,f} \text{ (поглощения не хватает)}, \\ A_{\max,f} - A_f < 0, & T_f < T_{\min,f} \text{ (поглощения слишком много)}, \\ 0, & \text{иначе.} \end{cases} \tag{18}$$

Разрешённый интервал значения A'_f после шага. Чтобы остаться в норме (или не ухудшить положение), значение A'_f должно лежать в интервале:

$$L_f = \min(A_f, A_{\min,f}), \quad U_f = \max(A_f, A_{\max,f}), \quad L_f \le A'_f \le U_f. \tag{19}$$

Так как $\Delta A_f = d_f \cdot S_3$ линейно по S_3, ограничение на площадь по частоте f принимает вид:

$$S_{\text{лим},f} = \begin{cases} (U_f - A_f)/d_f, & d_f > 0, \\ (A_f - L_f)/(-d_f), & d_f < 0, \\ +\infty, & d_f = 0. \end{cases} \tag{20}$$

Площадь размещаемого элемента. При автоматическом подборе:

$$S_3 = \min\left(S_{\text{своб}},\ S_{\text{треб}},\ \min_f S_{\text{лим},f}\right), \quad S_{\text{треб}} = \frac{|\Delta A_{\text{треб},f^*}|}{|d_{f^*}|}, \tag{21}$$

где f^* — худшая частота.

Пошаговый алгоритм. На каждом шаге:

1. Вычисляются T_f, δ_f, r и $\Delta A_{\text{треб},f}$ для трёх частот. Если $r = 0$ — рекомендации не требуются.

2. Определяется худшая частота $f^* = \arg\max_f(\delta_f / T_{\text{пр},f})$. Режим: добавление, если $T_{f^*} > T_{\max,f^*}$; удаление, если $T_{f^*} < T_{\min,f^*}$.

3. Оценка достижимости (режим добавления):

$$\Delta A_{\max} = \sum_p \max_e \max(0,\ \alpha_{e,f^*} - \alpha_{p,f^*}) \cdot S_{\text{своб},p}.$$

   Если $A_{f^*} + \Delta A_{\max} < A_{\min,f^*}$, норматив на f^* недостижим, выводится сообщение, алгоритм завершается. Режим удаления определяется аналогично.

4. Для каждой допустимой пары «элемент e — поверхность p» вычисляются d_f, S_3 и $\Delta A_{f^*} = d_{f^*} \cdot S_3$. Пара отбрасывается, если знак d_{f^*} не совпадает со знаком $\Delta A_{\text{треб},f^*}$ или $S_3 \le 0$.

5. Из оставшихся пар выбирается пара с максимальным по модулю приростом на худшей частоте:

$$(e^*, p^*) = \arg\max_{(e,p)} |\Delta A_{f^*}(e, p)|. \tag{22}$$

При равенстве выбирается элемент с меньшим идентификатором, затем поверхность по порядку: пол, потолок, стены 1–4.

6. Выбранный элемент размещается (или удаляется), обновляются $S_{\text{своб},p^*}$, A_f, T_f на всех частотах. Переход к шагу 1.

7. Если допустимых пар нет, а $r > 0$, выводится сообщение «по данной модели норматив не достигается» с указанием частот, оставшихся вне нормы, и итоговых значений T_f.

Максимальное количество сидячих мест.

$$N_{\max} = \left\lfloor \frac{S_{\text{пола}}}{S_{\text{места}}} \right\rfloor, \tag{23}$$

где $S_{\text{места}}$ — площадь на одно сидячее место с учётом проходов (м^2), берётся из справочника нормативов. Число мест, участвующих в расчёте поглощения, задаётся пользователем и не превышает $N_{\max}$.

2 Исходные данные и их обработка

2.1 Источник и формат входных данных

Исходными данными служат характеристики помещения, вводимые пользователем в интерактивном режиме или загружаемые из ранее сохранённой базы:

- геометрические размеры (длина, ширина, высота);
- перечень поверхностей с указанием материала отделки и площади;
- перечень элементов интерьера (ковры, шторы, занавесы, кресла, стулья, диваны, книжные шкафы) с их материалом, площадью и признаком возможности удаления/добавления;
- тип назначения помещения с нормативным диапазоном времени реверберации на трёх октавных частотах (125, 500 и 2000 Гц);
- количество и тип сидячих мест (занятых или пустых).

Дополнительно используются справочные CSV-файлы, содержащие коэффициенты звукопоглощения материалов и нормативные акустические параметры помещений. Источниками данных служат открытые инженерные таблицы:

https://frontacoustic.ru/blog/blog-page/koefficzientyi-zvukopoglosh коэффициенты звукопоглощения материалов.

https://pop-music.ru/articles/zvukoizolyatsiya-i-zvukopogloshchenie — коэффициенты звукопоглощения материалов.

https://indikotrade.ru/news/press-reliz/raschyet-akustiki-pomeshche?srsltid=AfmBOooWu_Zu5ASDtNYX3PbKYxJT4sDe49z5WY7fiJFbiJWw5soIP0yI — нормативы акустических параметров помещений.

https://studfile.net/preview/8894903/page:8/ — эквивалентная площадь звукопоглощения людей и кресел.

Формат CSV-файлов. Каждый файл содержит строку заголовка с именами колонок; разделитель — запятая; десятичный разделитель — точка;

кодировка — UTF-8. В заголовках материалов не допускаются запятые, чтобы не нарушать структуру таблицы.

Используются следующие файлы:

- `data/materials.csv` — материалы поверхностей (стены, пол, потолок): `material_name`, `alpha_125`, `alpha_500`, `alpha_2000`;
- `data/doors.csv` — материалы дверей: `door_name`, `alpha_125`, `alpha_500`, `alpha_2000`;
- `data/windows.csv` — материалы окон: `window_name`, `alpha_125`, `alpha_500`, `alpha_2000`;
- `data/wall_decor.csv` — декор на стену (ковры, полотна): `decor_name`, `alpha_125`, `alpha_500`, `alpha_2000`;
- `data/flour_carpet.csv` — декор на пол (ковры, ковровые покрытия, дорожки): `decor_name`, `alpha_125`, `alpha_500`, `alpha_2000`;
- `data/sofa.csv` — сидячие места: `seat_name`, `area_125`, `area_500`, `area_2000`, `floor_area`;
- `data/purpose.csv` — назначения помещений: `purpose_name`, `min_125`, `max_125`, `min_500`, `max_500`, `min_2000`, `max_2000`.

2.2 Модель предметной области: типы данных

Геометрия помещения. Геометрические параметры заданы записью `Dimensions`:

Листинг 1: Тип для хранения размеров помещения.

```
data Dimensions = Dimensions
    { roomLength :: Double
    , roomWidth :: Double
    , roomHeight :: Double
    } deriving (Show, Eq)
```

Спектр значений на трёх частотах. Все зависящие от частоты величины (коэффициенты поглощения, эквивалентные площади, время реверберации) представлены типом `Spectrum` — три значения на частотах 125, 500 и 2000 Гц:

Листинг 2: Спектр значений по трём частотам.

```haskell
data Spectrum = Spectrum
    { at125 :: Double
    , at500 :: Double
    , at2000 :: Double
    } deriving (Show, Eq)
```

Материалы. Материалы разделены по категориям в соответствии с разными CSV-файлами:

Листинг 3: Типы материалов.

```haskell
data SurfaceMaterial = SurfaceMaterial
    { surfaceMaterialName :: String
    , materialAbsorption :: Spectrum
    } deriving (Show, Eq)

data DoorMaterial = DoorMaterial
    { doorName      :: String
    , doorAbsorption :: Spectrum
    } deriving (Show, Eq)

data WindowMaterial = WindowMaterial
    { windowName     :: String
    , windowAbsorption :: Spectrum
    } deriving (Show, Eq)

data DecorMaterial = DecorMaterial
    { decorName     :: String
    , decorAbsorption :: Spectrum
    } deriving (Show, Eq)

data SeatMaterial = SeatMaterial
    { seatName           :: String
    , seatEquivalentArea :: Spectrum
    , seatFloorAreaPerUnit :: Double
```

```
    } deriving (Show, Eq)
```

Поверхности помещения и элементы интерьера. Каждая поверхность имеет идентификатор, материал, площадь и список элементов (двери, окна, декор):

Листинг 4: Поверхности и элементы интерьера.

```
data SurfaceId
    = Wall Int
    | Floor
    | Ceiling
    deriving (Show, Eq, Ord)

data Door = Door
    { doorId      :: String
    , doorMaterial :: DoorMaterial
    , doorArea    :: Double
    } deriving (Show, Eq)

data Window = Window
    { windowId      :: String
    , windowMaterial :: WindowMaterial
    , windowArea    :: Double
    } deriving (Show, Eq)

data DecorItem = DecorItem
    { decorMaterial :: DecorMaterial
    , decorArea    :: Double
    } deriving (Show, Eq)

data SurfaceElem
    = ElemDoor  Door
    | ElemWindow Window
    | ElemDecor DecorItem
    deriving (Show, Eq)

data Surface = Surface
    { surfaceId      :: SurfaceId
    , surfaceMaterial :: SurfaceMaterial
    , surfaceArea    :: Double
    , surfaceElems   :: [SurfaceElem]
```

```
    } deriving (Show, Eq)
```

Сидячие места. Сидячие места хранятся отдельно от поверхностей, так как всегда располагаются на полу:

Листинг 5: Сидячие места.

```
data SeatItem = SeatItem
    { seatCount   :: Int
    , seatMaterial :: SeatMaterial
    } deriving (Show, Eq)
```

Помещение. Само помещение объединяет название, геометрию, список поверхностей и список сидячих мест:

Листинг 6: Основной тип помещения.

```
data Room = Room
    { roomName      :: String
    , roomDimensions :: Dimensions
    , roomSurfaces  :: [Surface]
    , roomSeats     :: [SeatItem]
    } deriving (Show, Eq)
```

Тип назначения и нормативы. Для каждого типа назначения хранится диапазон $[T_{\min,f}, T_{\max,f}]$ на каждой из трёх частот:

Листинг 7: Назначение помещения и нормативы.

```
data Range = Range
    { rangeMin :: Double
    , rangeMax :: Double
    } deriving (Show, Eq)

data ReverbSpectrum = ReverbSpectrum
    { reverbLow :: Range
    , reverbMid :: Range
    , reverbHigh :: Range
    } deriving (Show, Eq)

data PurposeReq = PurposeReq
    { purposeName     :: String
```

```
    , recommendedReverb :: ReverbSpectrum
    } deriving (Show, Eq)
```

Результаты расчёта и оценки. Результат акустического расчёта хранит объём, общую площадь, суммарное поглощение и время реверберации; результат оценки — булев признак пригодности, отклонения по частотам и относительное отклонение:

Листинг 8: Результаты расчёта и оценки.

```
data AcousticResult = AcousticResult
    { roomVolume      :: Double
    , commonArea      :: Double
    , totalAbsorption :: Spectrum
    , reverberationTime :: Spectrum
    } deriving (Show, Eq)

data Deviation = Deviation
    { dev125 :: Double
    , dev500 :: Double
    , dev2000 :: Double
    } deriving (Show, Eq)

data Grade
    = Suitable
    | ConditionallySuitable
    | Unsuitable
    deriving (Show, Eq)

data SuitabilityResult = SuitabilityResult
    { suitable   :: Bool
    , deviations :: Deviation
    , relativeDev :: Double
    , grade      :: Grade
    , reachable  :: Bool
    } deriving (Show, Eq)
```

Иерархия ошибок. Все операции, способные завершиться неуспехом, возвращают тип `AppError`, разделяющий четыре класса нарушений:

Листинг 9: Иерархия ошибок приложения.

```
data AppError
    = FileError   String
    | ParseError  String
    | InvalidInput String
    | NotFound    String
    deriving (Show, Eq)
```

2.3 Хранение и загрузка помещений в формате JSON

Для геометрии помещения, материалов, помещения и спектра выведены экземпляры `FromJSON` и `ToJSON` (через библиотеку `aeson`), что позволяет сохранять и загружать базу описанных пользователем помещений без повторного ввода данных.

Модуль `Storage.JSON` инкапсулирует чтение и запись файла базы:

Листинг 10: Интерфейс модуля Storage.JSON.

```
loadRooms   :: FilePath -> IO (Either AppError [Room])
saveRooms   :: FilePath -> [Room] -> IO (Either AppError ())
addRoom     :: FilePath -> Room -> Bool -> IO (Either AppError ())
deleteRoom  :: FilePath -> Int -> IO (Either AppError ())

chooseExistingFileInDir :: FilePath -> IO (Either AppError FilePath)
chooseOrCreateFileInDir :: FilePath -> IO (Either AppError FilePath)
```

Особенности реализации:

- при отсутствии файла создаётся пустой список помещений;
- при добавлении помещения с существующим именем программа запрашивает у пользователя подтверждение перезаписи;

Листинг 11: Фрагмент сохранённого помещения.

```
[
    {
        "roomName": "Актовый зал",
        "roomDimensions": {
            "roomLength": 18.0,
            "roomWidth": 12.0,
```

```
            "roomHeight": 6.0
        },
        "roomSurfaces": [
            {
                "surfaceId": { "tag": "Wall", "contents": 1 },
                "surfaceMaterial": { ... },
                "surfaceArea": 108.0,
                "surfaceElems": [ ... ]
            },
            ...
        ],
        "roomSeats": [ ... ]
    }
]
```

Использование JSON в качестве формата хранения обусловлено необходимостью сохранять вложенную структуру помещения (поверхности, элементы интерьера, сидячие места), что неудобно делать в плоском CSV-формате. CSV применяется только для справочных данных, которые не изменяются пользователем в процессе работы программы.

3 Описание разработки проекта

3.1 Архитектура программного комплекса

Проект собирается системой **Stack** как библиотека `AcousticProject` с исполняемым модулем-точкой входа `app/Main.hs`. Логика приложения разделена на слои: типы данных предметной области, модули чтения входных данных (CSV и JSON), вычислительные модули (геометрия, поглощение, реверберация), модуль оценки пригодности, модуль формирования рекомендаций, модуль пакетной проверки, модули вывода отчёта, пользовательского ввода и логирования.

Всего в проекте **14 модулей**, организованных в иерархическую систему (таблица 1).

Таблица 1: Состав программного комплекса.

№	Назначение модуля	Файл
1	Типы данных предметной области	Types.*
2	Чтение CSV-файлов	Input.CSV
3	Чтение и запись JSON-файлов	Storage.JSON
4	Хранение данных о материалах и мебели	Types.Material
5	Хранение параметров помещения	Types.Room
6	Геометрические расчёты	Calculation.Geometry
7	Расчёт эквивалентной площади звукопоглощения	Calculation.Absorption
8	Расчёт времени реверберации	Calculation.Reverberation
9	Оценка пригодности помещения	Evaluation.Suitability
10	Формирование рекомендаций	Recommendation.Acoustic
11	Пакетная проверка помещений	Batch.Check
12	Формирование отчёта	Output.Report
13	Пользовательский ввод	Input.CLI
14	Логирование	Log.Logger

3.1.1 Модуль Types

Пакет модулей `Types.*` (`Types.Spectrum`, `Types.Geometry`, `Types.Material`, `Types.Room`, `Types.Purpose`, `Types.AcousticResult`, `Types.Suitability`, `Types.Recommendation`, `Types.Report`, `Types.AppError`) не зависит от вычислительной логики и определяет всю систему типов проекта. Иерархия ошибок `AppError` разделяет четыре класса нарушений: ошибки файловой системы, ошибки разбора данных, некорректный ввод пользователя и ситуацию «не найдено» (см. листинг 9).

3.1.2 Модуль Input.CSV

Модуль `Input.CSV` реализует чтение всех CSV-файлов через библиотеку **cassava**. Для каждого типа данных определён экземпляр класса `FromNamedRecord`, что позволяет использовать одну общую функцию загрузки:

Листинг 12: Общая функция загрузки CSV.

```
loadCsv :: FromNamedRecord a => FilePath -> IO (Either AppError [a])
loadCsv filePath = do
    readResult <- try (BL.readFile filePath)
    case readResult of
        Left err -> pure (Left (FileError (show err)))
        Right content ->
            case decodeByName content of
                Left err -> pure (Left (ParseError err))
                Right (_, records) -> pure (Right (V.toList records))
```

Публичные функции модуля: `loadSurfaceMaterials`, `loadDoorMaterials`, `loadWindowMaterials`, `loadWallDecor`, `loadFloorDecor`, `loadSeatMaterials`, `loadPurpose`.

3.1.3 Модуль Calculation.Geometry

Модуль реализует расчёт объёма, площадей поверхностей и свободных площадей, сидячих мест, полную площадь, площади без сидячих мест, а также проверку того, что площадь сидячих мест меньше, чем площадь пола и что свободная площадь больше или равна нулю:

Листинг 13: Геометрические расчёты.

```haskell
calculateVolume :: Dimensions -> Double
calculateVolume d =
    roomLength d * roomWidth d * roomHeight d

calculateFloorArea :: Dimensions -> Double
calculateFloorArea d =
    roomLength d * roomWidth d

calculateTotalSurfaceArea :: Dimensions -> Double
calculateTotalSurfaceArea d =
    2 * ( roomLength d * roomWidth d
        + roomLength d * roomHeight d
        + roomWidth d * roomHeight d
        )

calculateSeatsArea :: [SeatItem] -> Double
calculateSeatsArea seats =
    sum [ fromIntegral (seatCount s)
          * seatFloorAreaPerUnit (seatMaterial s)
        | s <- seats ]

calculateCommonArea :: Dimensions -> [SeatItem] -> Double
calculateCommonArea d seats =
    calculateTotalSurfaceArea d - calculateSeatsArea seats

calculateFreeArea :: Surface -> [SeatItem] -> Double
calculateFreeArea surface seats =
    let baseArea    = surfaceArea surface
        elemsArea   = sum (map elemArea (surfaceElems surface))
        seatsOnFloor = case surfaceId surface of
                         Floor -> calculateSeatsArea seats
                         _     -> 0
    in baseArea - elemsArea - seatsOnFloor

elemArea :: SurfaceElem -> Double
elemArea (ElemDoor d) = doorArea d
elemArea (ElemWindow w) = windowArea w
elemArea (ElemDecor d) = decorArea d

validateSeatsArea :: Dimensions -> [SeatItem] -> Either String ()
```

```
validateSeatsArea d seats
    | calculateSeatsArea seats > calculateFloorArea d =
        Left "Площадь зонысидячихместпревышаетплощадьпола"
    | otherwise = Right ()

validateFreeArea :: Surface -> [SeatItem] -> Either String ()
validateFreeArea surface seats
    | calculateFreeArea surface seats < 0 =
        Left ("Свободная площадьповерхностиотрицательна: "
              ++ show (surfaceId surface))
    | otherwise = Right ()
```

3.1.4 Модуль Calculation.Absorption

Модуль реализует расчёт эквивалентной площади звукопоглощения. Учитывается вклад поверхностей (с вычетом закрытой площади), мебели, сидячих мест и добавочного поглощения:

Листинг 14: Расчёт эквивалентной площади звукопоглощения.

```
calculateSurfaceAbsorption :: Surface -> [SeatItem] -> Spectrum
calculateSurfaceAbsorption surface seats =
    addSpectrums basePart elemPart
  where
    freeArea = calculateFreeArea surface seats
    baseAlpha = materialAbsorption (surfaceMaterial surface)
    basePart = multiplySpectrum baseAlpha freeArea
    elemPart = sumSpectrums (map elemAbsorption (surfaceElems surface))

    elemAbsorption (ElemDoor d) =
        multiplySpectrum (doorAbsorption (doorMaterial d)) (doorArea d)
    elemAbsorption (ElemWindow w) =
        multiplySpectrum (windowAbsorption (windowMaterial w)) (windowArea
            w)
    elemAbsorption (ElemDecor d) =
        multiplySpectrum (decorAbsorption (decorMaterial d)) (decorArea d)

calculateSurfacesAbsorption :: [Surface] -> [SeatItem] -> Spectrum
calculateSurfacesAbsorption surfaces seats =
    sumSpectrums
        [ calculateSurfaceAbsorption s seats
        | s <- surfaces ]
```

```
22
23 calculateSeatsAbsorption :: [SeatItem] -> Spectrum
24 calculateSeatsAbsorption seats =
25     sumSpectrums
26         [ multiplySpectrum
27             (seatEquivalentArea (seatMaterial s))
28             (fromIntegral (seatCount s))
29         | s <- seats ]
30
31 additionalAlpha :: Spectrum
32 additionalAlpha = Spectrum
33     { at125 = 0.0425
34     , at500 = 0.0225
35     , at2000 = 0.0225
36     }
37
38 calculateAdditionalAbsorption :: Double -> Spectrum
39 calculateAdditionalAbsorption commonArea =
40     multiplySpectrum additionalAlpha commonArea
41
42 calculateTotalAbsorption
43     :: [Surface]
44     -> [SeatItem]
45     -> Double
46     -> Spectrum
47 calculateTotalAbsorption surfaces seats commonArea =
48     addSpectrums
49         (addSpectrums
50             (calculateSurfacesAbsorption surfaces seats)
51             (calculateSeatsAbsorption seats))
52         (calculateAdditionalAbsorption commonArea)
```

3.1.5 Модуль Calculation.Reverberation

Модуль реализует расчёт времени реверберации по формуле Сэбина или Эйринга в зависимости от среднего коэффициента звукопоглощения. Выбор формулы выполняется **отдельно для каждой частоты**:

Листинг 15: Расчёт времени реверберации.

```
1 sabineConstant :: Double
2 sabineConstant = 0.163
```

```haskell

calculateSabine :: Double -> Double -> Double
calculateSabine volume totalAbsorption
    | totalAbsorption <= 0 = 0
    | otherwise          = sabineConstant * volume / totalAbsorption

calculateEyring :: Double -> Double -> Double -> Double
calculateEyring volume commonArea phi
    | commonArea <= 0 = 0
    | phi <= 0       = 0
    | otherwise      = sabineConstant * volume / (commonArea * phi)

calculateReverberationTime
    :: Double
    -> Double
    -> Double
    -> Double
calculateReverberationTime volume totalAbsorption commonArea
    | totalAbsorption <= 0 = 0
    | commonArea <= 0    = 0
    | alphaAvg <= 0.2    = calculateSabine volume totalAbsorption
    | alphaAvg >= 1.0    = 0
    | otherwise          = calculateEyring volume commonArea phi
  where
    alphaAvg = totalAbsorption / commonArea
    phi      = negate (log (1 - alphaAvg))

calculateReverberationSpectrum
    :: Double
    -> Spectrum
    -> Double
    -> Spectrum
calculateReverberationSpectrum volume absorptions commonArea =
    Spectrum
        { at125 = calculateReverberationTime volume (at125 absorptions)
            commonArea
        , at500 = calculateReverberationTime volume (at500 absorptions)
            commonArea
        , at2000 = calculateReverberationTime volume (at2000 absorptions)
            commonArea
        }
```

3.1.6 Модуль Evaluation.Suitability

Модуль сравнивает рассчитанное время реверберации с рекомендуемым диапазоном выбранного назначения и формирует `SuitabilityResult`:

Листинг 16: Оценка акустической пригодности.

```
calculateDeviation :: Double -> Range -> Double
calculateDeviation value (Range lo hi)
    | value < lo = lo - value
    | value > hi = value - hi
    | otherwise = 0

calculateDeviations :: Spectrum -> ReverbSpectrum -> Deviation
calculateDeviations t spectrum =
    Deviation
        { dev125 = calculateDeviation (at125 t) (reverbLow spectrum)
        , dev500 = calculateDeviation (at500 t) (reverbMid spectrum)
        , dev2000 = calculateDeviation (at2000 t) (reverbHigh spectrum)
        }

calculateRelativeDeviation :: Spectrum -> ReverbSpectrum -> Double
calculateRelativeDeviation t spectrum =
    maximum
        [ relativeAt (at125 t) (reverbLow spectrum)
        , relativeAt (at500 t) (reverbMid spectrum)
        , relativeAt (at2000 t) (reverbHigh spectrum)
        , 0
        ]
  where
    relativeAt value (Range lo hi)
        | value < lo = (lo - value) / lo
        | value > hi = (value - hi) / hi
        | otherwise = 0

gradeFromDeviation :: Double -> Grade
gradeFromDeviation r
    | r == 0   = Suitable
    | r <= 0.1 = ConditionallySuitable
    | otherwise = Unsuitable

evaluateSuitability
    :: AcousticResult
```

```
    -> PurposeReq
    -> SuitabilityResult
evaluateSuitability acoustic purpose =
    SuitabilityResult
        { suitable   = grade == Suitable
        , deviations = devs
        , relativeDev = relDev
        , grade      = grade
        , reachable  = True
        }
  where
    t       = reverberationTime acoustic
    spectrum = recommendedReverb purpose
    devs    = calculateDeviations t spectrum
    relDev  = calculateRelativeDeviation t spectrum
    grade   = gradeFromDeviation relDev
```

3.1.7 Модуль Recommendation.Acoustic

Модуль реализует жадный многошаговый алгоритм формирования рекомендаций. Точка входа — `generateRecommendations`:

Листинг 17: Точка входа рекомендательной системы.

```
generateRecommendations
    :: Room
    -> PurposeReq
    -> AcousticResult
    -> SuitabilityResult
    -> [DecorMaterial]
    -> [DecorMaterial]
    -> [Recommendation]
generateRecommendations room purpose acoustic suitability wallCat
    floorCat
    | suitable suitability = []
    | not (isAchievable room purpose acoustic (wallCat ++ floorCat)) = []
    | otherwise = recommendLoop room purpose acoustic suitability
                                wallCat floorCat []
```

Основные компоненты модуля:

- `findWorstFrequency` — определение худшей частоты f^* по относительному отклонению;

- `chooseBestPair` — выбор пары «элемент — поверхность» по максимуму прироста на худшей частоте;
- `areaLimitAt`, `limitArea` — расчёт лимитной площади по формуле (19);
- `requiredArea` — расчёт требуемой площади по формуле (20);
- `applyChange`, `applyRecommendations` — применение изменений к помещению;
- `recalculateAcoustic` — пересчёт акустики после изменения;
- `isAchievable` — проверка достижимости норматива.

При исчерпании элементов каталога функция возвращает пустой список; сообщение «по данной модели норматив не достигается» выводится в модуле `Input.CLI`.

3.1.8 Модуль Output.Report

Модуль формирует два вида отчётов:

- `generateReport` — полный отчёт по одному помещению (секции: помещение, назначение, акустика до рекомендаций, оценка, рекомендации, акустика после, предупреждение, пометка об оценочном характере расчёта);
- `generateSummaryReport` — сводный отчёт по пакету помещений.

Отчёты сохраняются в текстовые файлы функциями `saveReport` и `saveSummaryReport`. Секция «Акустика после рекомендаций» выводится только при наличии хотя бы одной рекомендации, что устраняет дублирование данных в отчёте.

3.1.9 Модуль Storage.JSON

Модуль инкапсулирует чтение и запись файла базы помещений. Публичный интерфейс:

Листинг 18: Интерфейс модуля Storage.JSON.

```
loadRooms   :: FilePath -> IO (Either AppError [Room])
saveRooms   :: FilePath -> [Room] -> IO (Either AppError ())
addRoom     :: FilePath -> Room -> Bool -> IO (Either AppError ())
deleteRoom  :: FilePath -> Int -> IO (Either AppError ())

chooseExistingFileInDir :: FilePath -> IO (Either AppError FilePath)
chooseOrCreateFileInDir :: FilePath -> IO (Either AppError FilePath)
```

Особенности реализации:

- при отсутствии файла создаётся пустой список помещений;
- при добавлении помещения с существующим именем программа запрашивает подтверждение перезаписи;
- типы с несколькими вариантами значения (`SurfaceId`, `SurfaceElem`) сохраняются в JSON в виде объекта с полями `tag` (название варианта) и `contents` (данные внутри варианта). Это позволяет однозначно восстановить исходную структуру при загрузке. Например, значение «стена номер 1» записывается как `{"tag": "Wall "contents": 1}`, а «пол» — как `{"tag": "Floor"}`.

3.1.10 Модуль Batch.Check

Модуль реализует пакетную проверку нескольких помещений, загружаемых из JSON-файла, с формированием сводного отчёта. Невалидные помещения пропускаются с записью в лог.

Листинг 19: Пакетная проверка помещений.

```
checkAllRooms
    :: FilePath -> PurposeReq -> FilePath -> FilePath -> FilePath
    -> IO (Either AppError BatchResult)
checkAllRooms roomsFile purpose wallFile floorFile logPath = do
    roomsResult <- loadRooms roomsFile
    case roomsResult of
        Left err -> pure (Left err)
        Right rooms -> do
            wallResult <- loadWallDecor wallFile
            case wallResult of
```

```
            Left err -> pure (Left err)
            Right wallCatalog -> do
                floorResult <- loadFloorDecor floorFile
                case floorResult of
                    Left err -> pure (Left err)
                    Right floorCatalog -> do
                        entries <- forM rooms $ \room ->
                            case validateRoomForBatch room of
                                Left reason -> do
                                    _ <- logWarning logPath
                                            ("Пропущено: " ++ roomName
                                               room
                                             ++ " --" ++ reason)
                                    pure (BatchSkipped (roomName room)
                                       reason)
                                Right () ->
                                    case checkOneRoom wallCatalog
                                                      floorCatalog
                                                      room purpose of
                                        Left err -> ... BatchSkipped ...
                                        Right report -> ... BatchSuccess
                                            ...
                        ...
                        pure (Right result)
```

3.1.11 Модуль Input.CLI

Модуль реализует консольное меню из пяти пунктов:

1. `Создать и проверить помещение`
2. `Создать и сохранить помещение`
3. `Пакетная проверка из файла`
4. `Просмотреть сохранённые помещения`
5. `Удалить помещение`
0. `Выход`

Все вводимые значения проходят валидацию. Функции `readInt`, `readDouble`, `readPositiveInt`, `readPositiveDouble`, `readPositiveDoubleInRange`, `readInRange`, `readRoomName` при некорректном вводе выводят сообщение и записывают предупреждение в журнал через `logWarning`.

При создании помещения пользователь последовательно вводит имя, размеры, материалы для шести поверхностей, окна, двери, элементы декора и сидячие места. Для окон, дверей и декора выполняется привязка к конкретной поверхности и ограничение площади оставшейся свободной площадью; для сидячих мест ведётся отдельный счётчик оставшейся площади пола.

3.1.12 Модуль Log.Logger

Модуль реализует четыре уровня логирования — `Debug`, `Info`, `Warning`, `Error`. Каждая запись сопровождается меткой времени, полученной через библиотеку **time**:

Листинг 20: Основная функция логирования.

```
logMessage
    :: FilePath -> LogLevel -> String -> IO (Either AppError ())
logMessage filePath level message = do
    timestamp <- getCurrentTime
    let timeStr = formatTime defaultTimeLocale "%Y-%m-%d %H:%M:%S"
        timestamp
        entry  = "[" ++ timeStr ++ "] ["
                    ++ levelText level ++ "] "
                    ++ message ++ "\n"
    result <- try (appendFile filePath entry)
    case result of
        Left err -> pure (Left (FileError (show err)))
        Right () -> pure (Right ())
```

3.2 Технические особенности реализации

3.2.1 Единицы измерения и представление данных

Все геометрические величины выражены в метрах, площади — в квадратных метрах, объём — в кубических метрах, время реверберации — в секундах. Коэффициенты звукопоглощения безразмерны и лежат в диапазоне $[0, 1]$. Такое единообразие устраняет необходимость промежуточных пересчётов и упрощает сопоставление с табличными справочными значениями строительной акустики.

3.2.2 Обработка ошибок через Either

Все операции, способные завершиться неуспехом (чтение и запись базы помещений, разбор JSON и CSV, валидация пользовательского ввода), возвращают `Either AppError`. Это исключает аварийное завершение программы: ошибка является обычным значением, которое распространяется по типу и обрабатывается в одной точке — функции `printError` в модуле `Input.CLI`.

Разделение `AppError` на классы (файловые ошибки, ошибки разбора, некорректный ввод, «не найдено») позволяет формировать осмысленные диагностические сообщения.

3.2.3 Логирование действий и событий программы

Логирование распределено по модулям:

- `Main.hs` — фиксирует старт и завершение программы, а также критические ошибки;
- `Input.CLI` — фиксирует действия пользователя: некорректный ввод, создание и сохранение помещений, результаты проверки пригодности, ошибки сохранения отчёта;
- `Batch.Check` — фиксирует события пакетной проверки: обработанные помещения (Info), пропущенные из-за невалидности (Warning), ошибки проверки (Error).

Файл журнала `data/app.log` содержит метки времени и уровень важности для каждой записи, что обеспечивает прослеживаемость всех действий пользователя.

3.3 Тестирование

Корректность вычислительных функций подтверждена тестами свойств на основе библиотеки **QuickCheck**. Для каждого свойства генерируются случайные входные данные, и проверяется выполнение инварианта. Тесты организованы в модули, зеркально повторяющие структуру библиотеки:

- `Calculation.GeometrySpec` — 4 свойства: неотрицательность объёма, точная формула объёма, монотонность по длине, превышение полной площади поверхностей над площадью пола;

- `Calculation.ReverberationSpec` — 11 свойств: для формулы Сэбина (неотрицательность, нуль при нулевом поглощении, точная формула, пропорциональность объёму) и для формулы Эйринга (неотрицательность, нулевые значения, точная формула, пропорциональности объёму, площади и функции φ);

- `Types.SpectrumSpec` — 4 свойства: коммутативность сложения, нейтральный элемент, умножение на 1, вычитание из себя;

- `Evaluation.SuitabilitySpec` — 3 свойства: неотрицательность отклонения, нулевое отклонение внутри диапазона, оценка «пригодно» при $r = 0$.

Точка входа `test/Spec.hs` последовательно запускает все свойства, группируя вывод по модулям. Запуск — командой `stack test`. Каждое свойство проверяется на 100 случайных наборах входных данных.

4 Примеры работы программы

4.1 Создание помещения и проверка на пригодность

Сценарий: пользователь создаёт актовый зал 18×12×6 м, выбирает материалы поверхностей, добавляет 1 элемент интерьера.

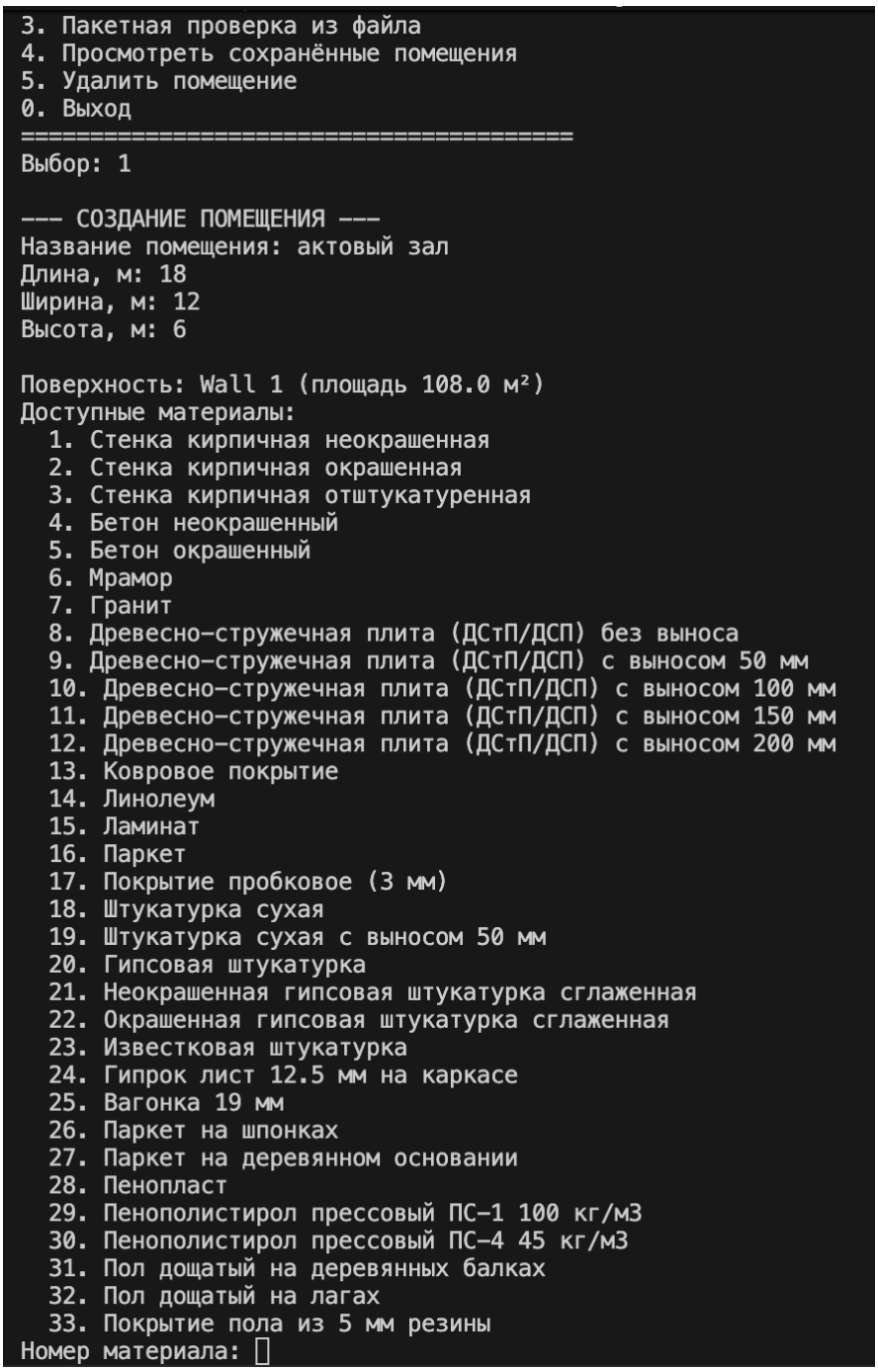


Рис. 1: Создание помещения(1)

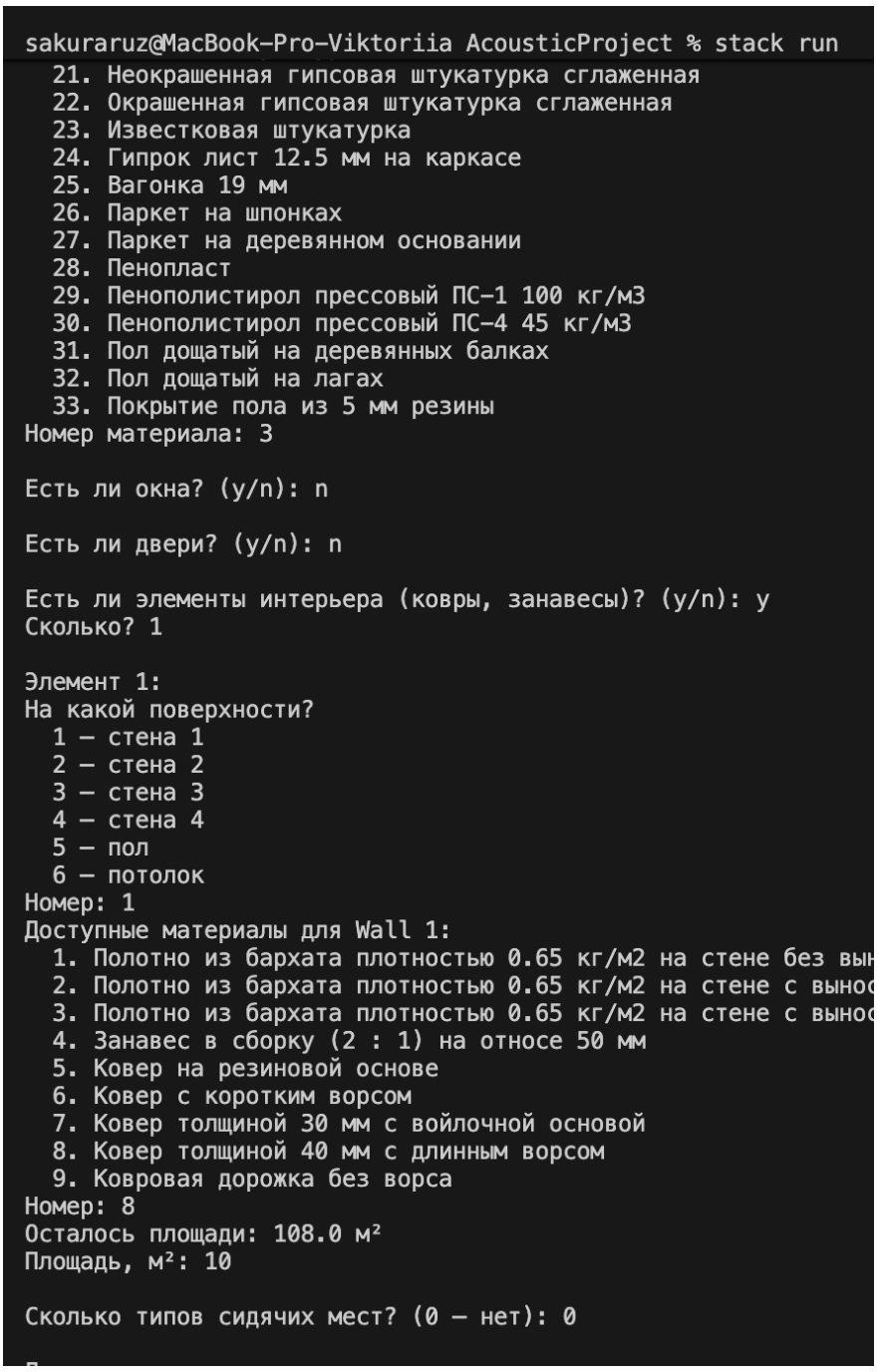


Рис. 2: Создание помещения(2)

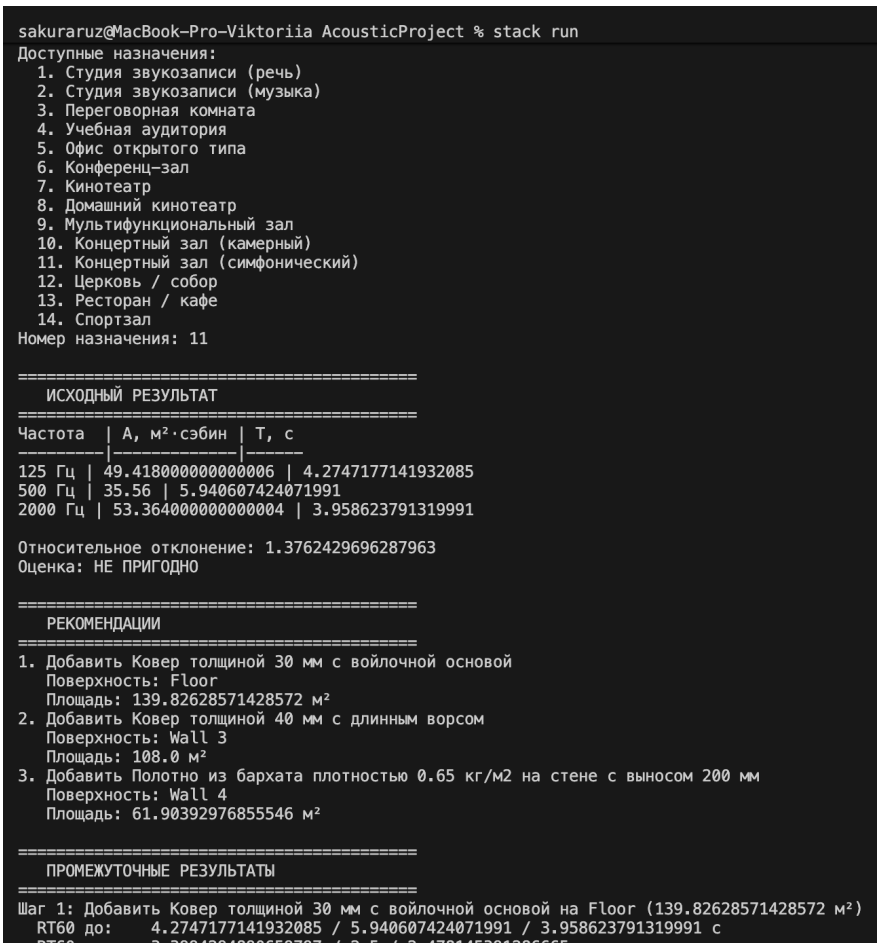


Рис. 3: Создание помещения(3)

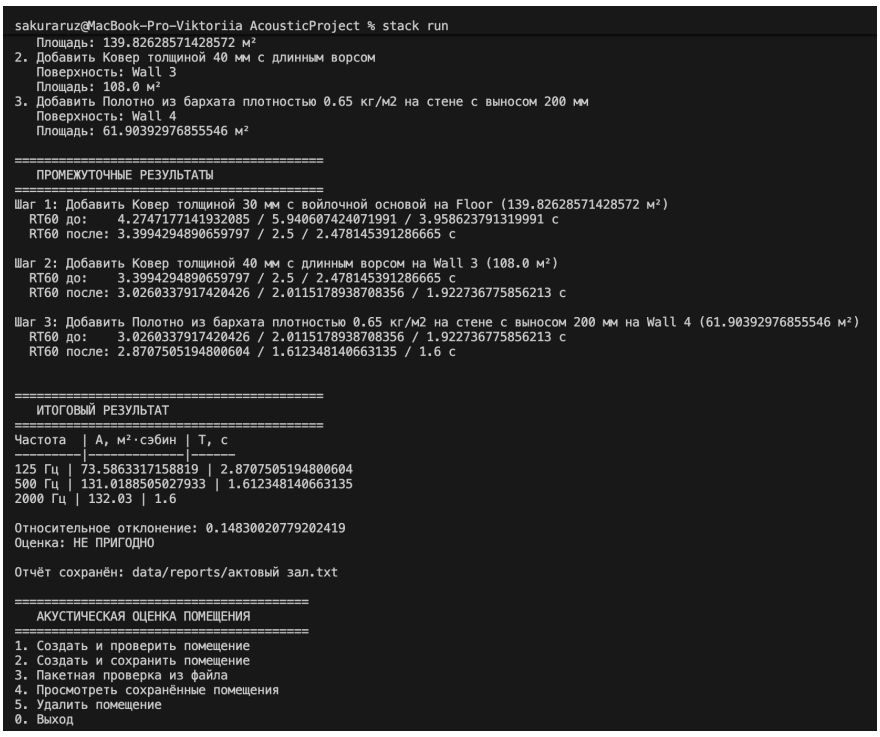


Рис. 4: Создание помещения(4)

4.2 Создание и запись в файл

По выбору пункта 2 можно создать комнату и записать в существующий или в новый JSON файл.

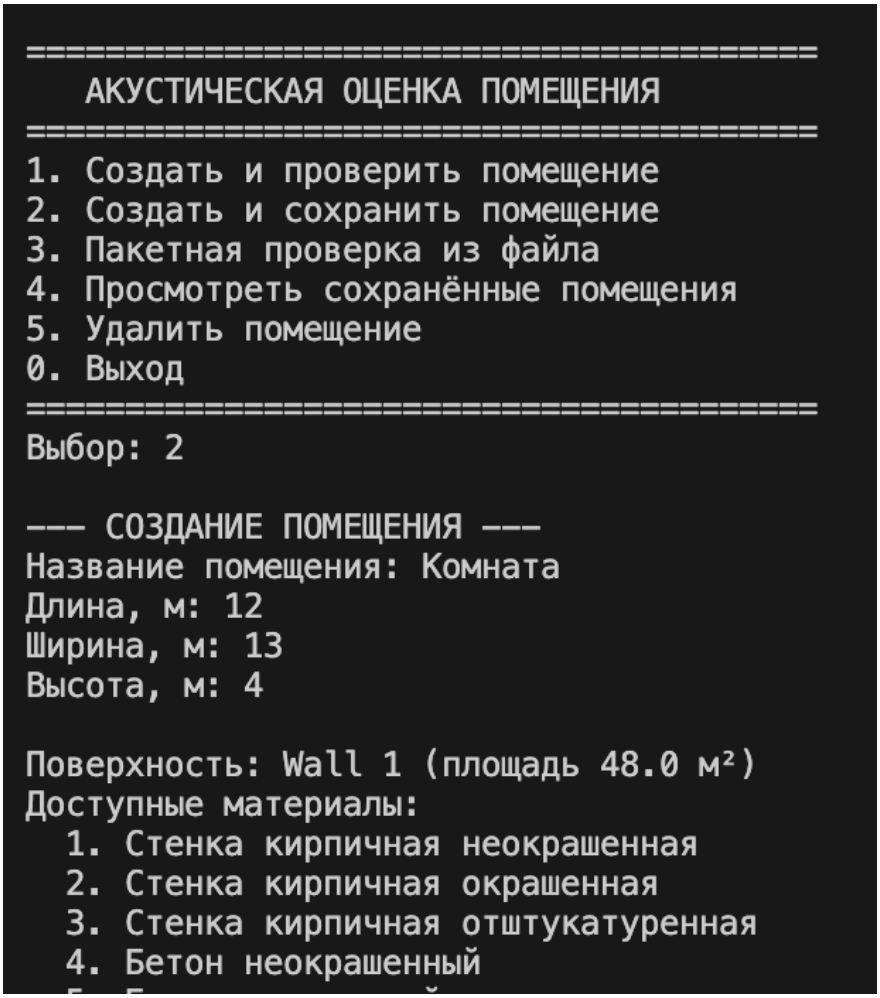


Рис. 5: Создание помещения(1)

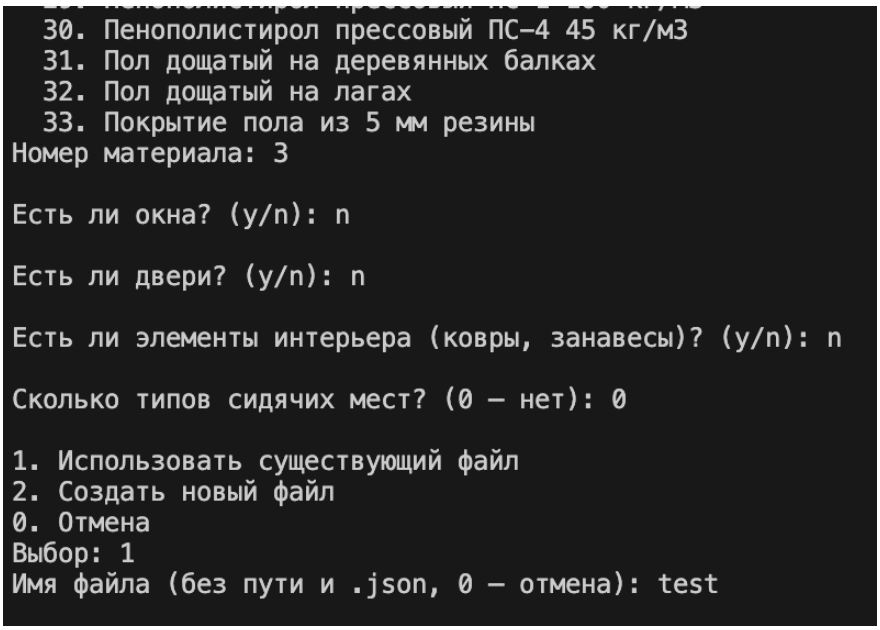


Рис. 6: Создание помещения(2)

4.3 Пакетная проверка из файла

По выбору пункта 3 программа спрашивает имя файла, которого надо проверить и после просит ввести назначение, по которому будем проверять помещения.

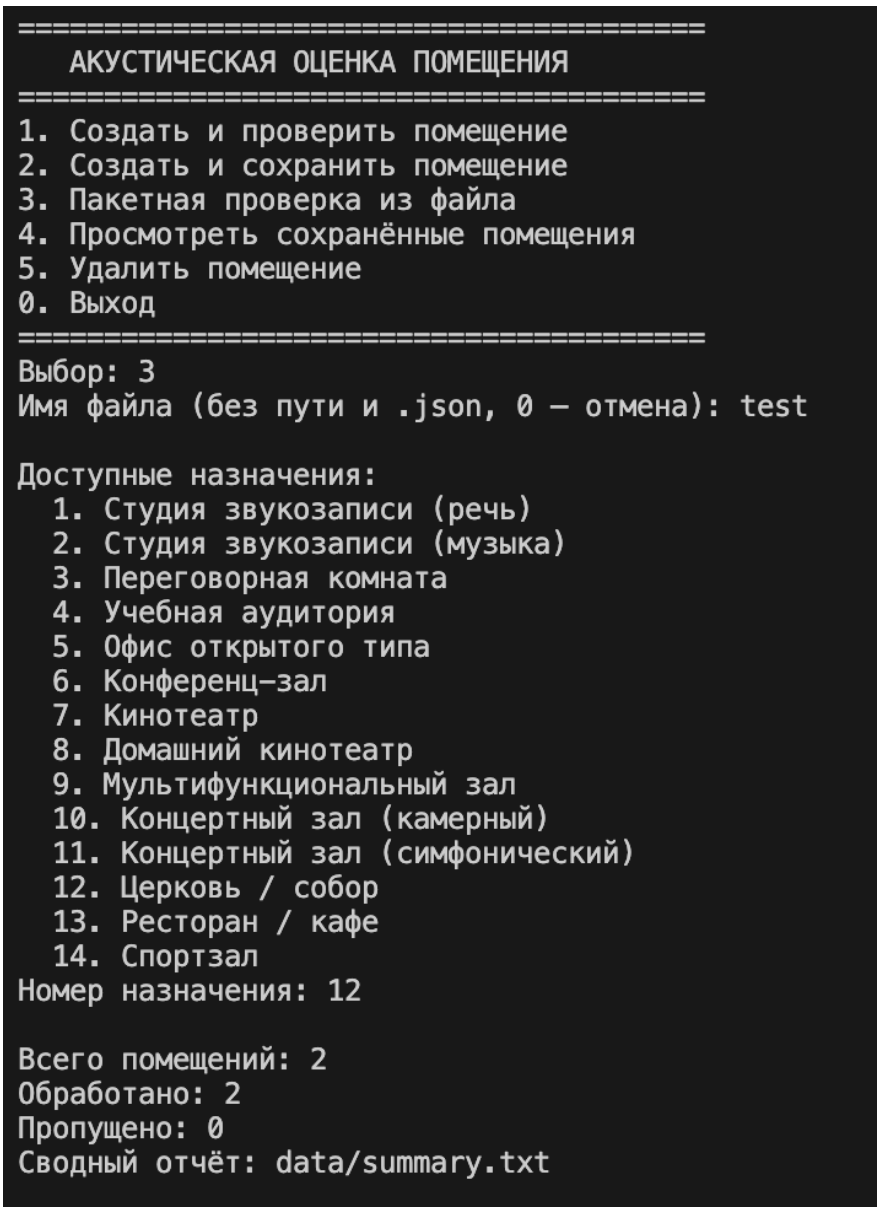


Рис. 7: Пакетная проверка из файла

Сводный отчёт записывается в отдельный текстовый файл.

4.4 Просмотр сохранённых помещений

С помощью пункта 4 можно посмотреть названия помещений в определённом JSON файле.

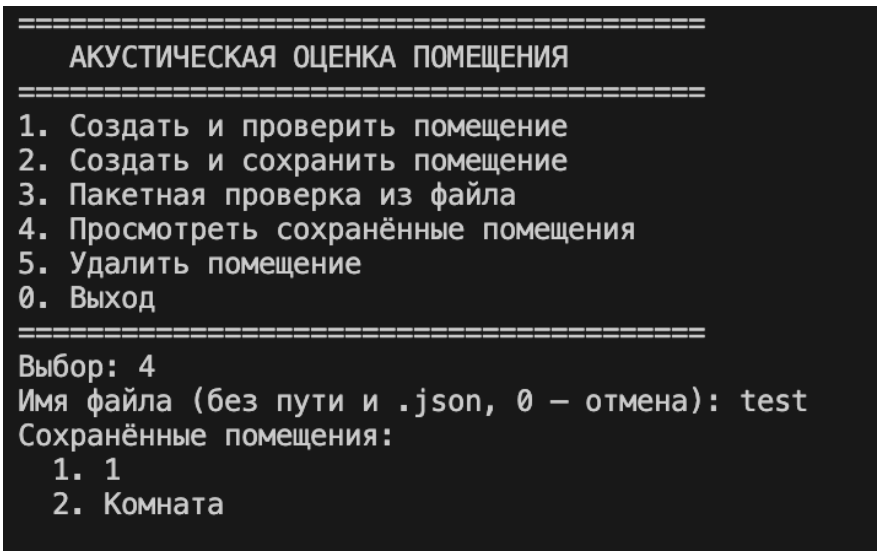


Рис. 8: Просмотр сохранённых помещений

4.5 Удаление помещения

При нажатии пункта 5 можно удалить выбранное помещение из файла.

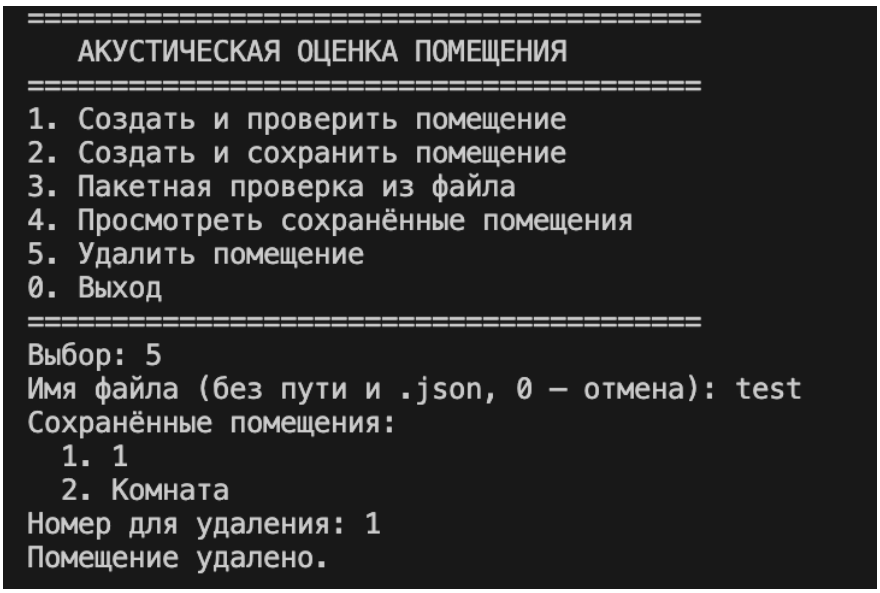


Рис. 9: Удаление помещения

4.6 Обработка ошибок

При некорректном пользовательском вводе программа просит повторить попытку, а в лог записывается, что пользователь сделал неправильный ввод

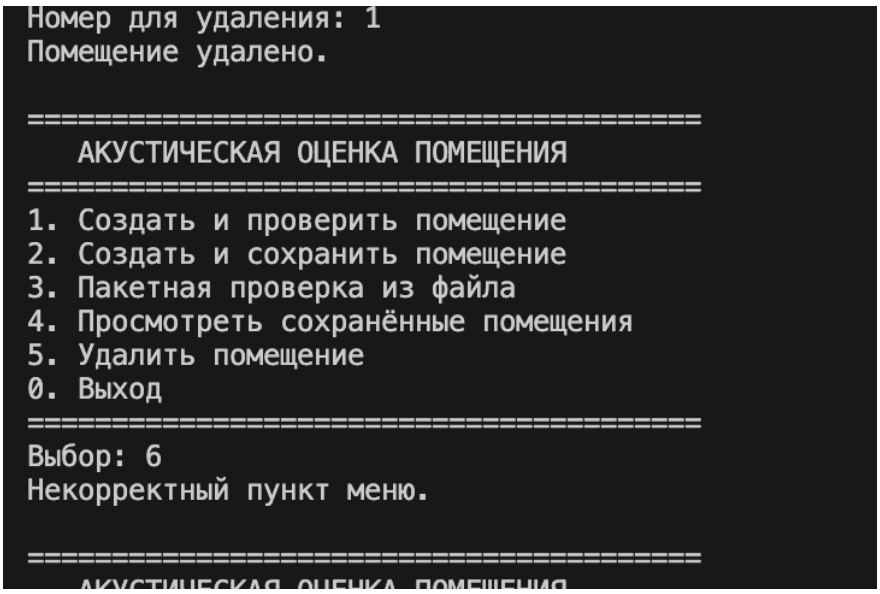


Рис. 10: Обработка ошибок

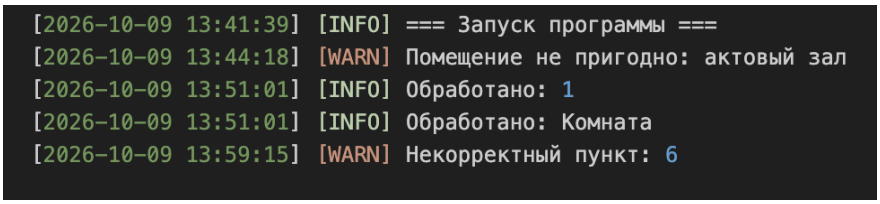


Рис. 11: Запись в лог

5 Руководство по эксплуатации

5.1 Требования к окружению

Для сборки и запуска программы необходимы:

- компилятор **GHC** версии 9.10.3 (или совместимой);
- система сборки **Stack**;
- операционная система с поддержкой консольного ввода-вывода (в рамках работы проверялась на **macOS**, но программа переносима на Linux и Windows);
- библиотеки языка Haskell, указанные в файле `package.yaml`: `cassava`, `aeson`, `aeson-pretty`, `bytestring`, `vector`, `time`, `directory`, `QuickCheck`.

Для работы программы необходимы входные файлы с данными о материалах и назначениях помещений в формате CSV. Сведения о сохранённых помещениях хранятся в файле JSON. Результаты работы программы сохраняются в текстовые файлы.

Структура используемых файлов данных:

- `data/materials.csv` — справочник материалов поверхностей и их коэффициентов звукопоглощения;
- `data/doors.csv` — справочник материалов дверей;
- `data/windows.csv` — справочник материалов окон;
- `data/wall_decor.csv` — справочник декора на стену;
- `data/flour_carpet.csv` — справочник декора на пол;
- `data/sofa.csv` — справочник сидячих мест;
- `data/purpose.csv` — справочник типов назначения помещений и нормативных значений времени реверберации;
- `data/rooms/` — директория для сохранённых помещений в формате JSON;
- `data/reports/` — директория для отчётов;
- `data/summary.txt` — сводный отчёт по пакетной проверке;
- `data/app.log` — журнал работы программы.

5.2 Запуск программы

Для запуска программы необходимо перейти в корневую директорию проекта и выполнить команду:

```
stack run
```

После первого запуска программа создаёт журнал `data/app.log` и выводит главное меню:

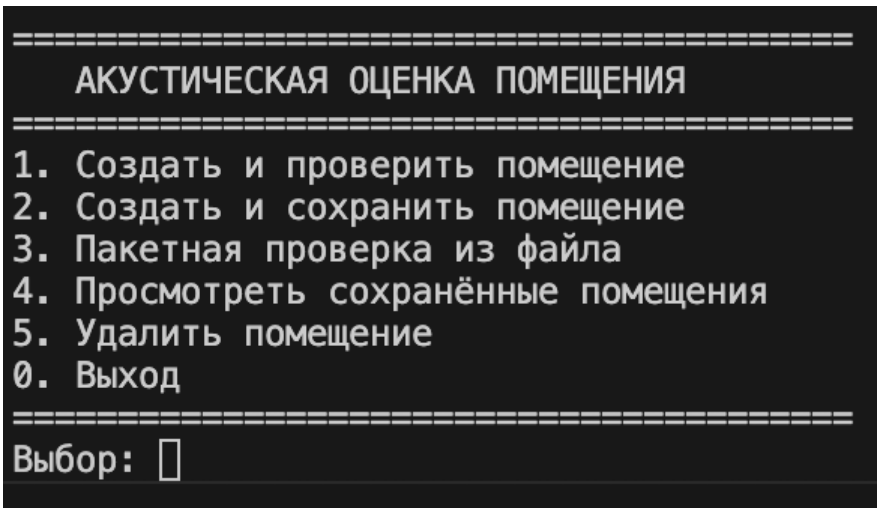


Рис. 12: Главное меню программы

Пользователь выбирает действие, вводя номер соответствующего пункта меню. Меню содержит следующие пункты:

1. Создать и проверить помещение.
2. Создать и сохранить помещение.
3. Пакетная проверка из файла.
4. Просмотреть сохранённые помещения.
5. Удалить помещение.
6. Выход.

5.3 Создание и проверка помещения

Для создания нового помещения и его немедленной проверки без сохранения необходимо выбрать пункт 1 главного меню.

5.3.1 Ввод названия и размеров

На первом этапе программа запрашивает название помещения, а затем его геометрические размеры — длину, ширину и высоту в метрах. Все размеры должны быть положительными числами.

```
3. Пакетная проверка из файла
4. Просмотреть сохранённые помещения
5. Удалить помещение
0. Выход
========================================
Выбор: 1

--- СОЗДАНИЕ ПОМЕЩЕНИЯ ---
Название помещения: актовый зал
Длина, м: 18
Ширина, м: 12
Высота, м: 6

Поверхность: Wall 1 (площадь 108.0 м²)
Доступные материалы:
  1. Стенка кирпичная неокрашенная
  2. Стенка кирпичная окрашенная
  3. Стенка кирпичная отштукатуренная
  4. Бетон неокрашенный
  5. Бетон окрашенный
  6. Мрамор
  7. Гранит
  8. Древесно-стружечная плита (ДСтП/ДСП) без выноса
  9. Древесно-стружечная плита (ДСтП/ДСП) с выносом 50 мм
  10. Древесно-стружечная плита (ДСтП/ДСП) с выносом 100 мм
  11. Древесно-стружечная плита (ДСтП/ДСП) с выносом 150 мм
  12. Древесно-стружечная плита (ДСтП/ДСП) с выносом 200 мм
  13. Ковровое покрытие
  14. Линолеум
  15. Ламинат
  16. Паркет
  17. Покрытие пробковое (3 мм)
  18. Штукатурка сухая
  19. Штукатурка сухая с выносом 50 мм
  20. Гипсовая штукатурка
  21. Неокрашенная гипсовая штукатурка сглаженная
  22. Окрашенная гипсовая штукатурка сглаженная
  23. Известковая штукатурка
  24. Гипрок лист 12.5 мм на каркасе
  25. Вагонка 19 мм
  26. Паркет на шпонках
  27. Паркет на деревянном основании
  28. Пенопласт
  29. Пенополистирол прессовый ПС-1 100 кг/м3
  30. Пенополистирол прессовый ПС-4 45 кг/м3
  31. Пол дощатый на деревянных балках
  32. Пол дощатый на лагах
  33. Покрытие пола из 5 мм резины
Номер материала:
```

Рис. 13: Ввод названия и размеров помещения

5.3.2 Выбор материалов поверхностей

Далее для каждой из шести поверхностей (четырёх стен, пола и потолка) программа выводит список доступных материалов с указанием их коэффициентов звукопоглощения. Пользователь выбирает материал по его номеру. Площадь каждой поверхности программа рассчитывает автоматически по введённым размерам.

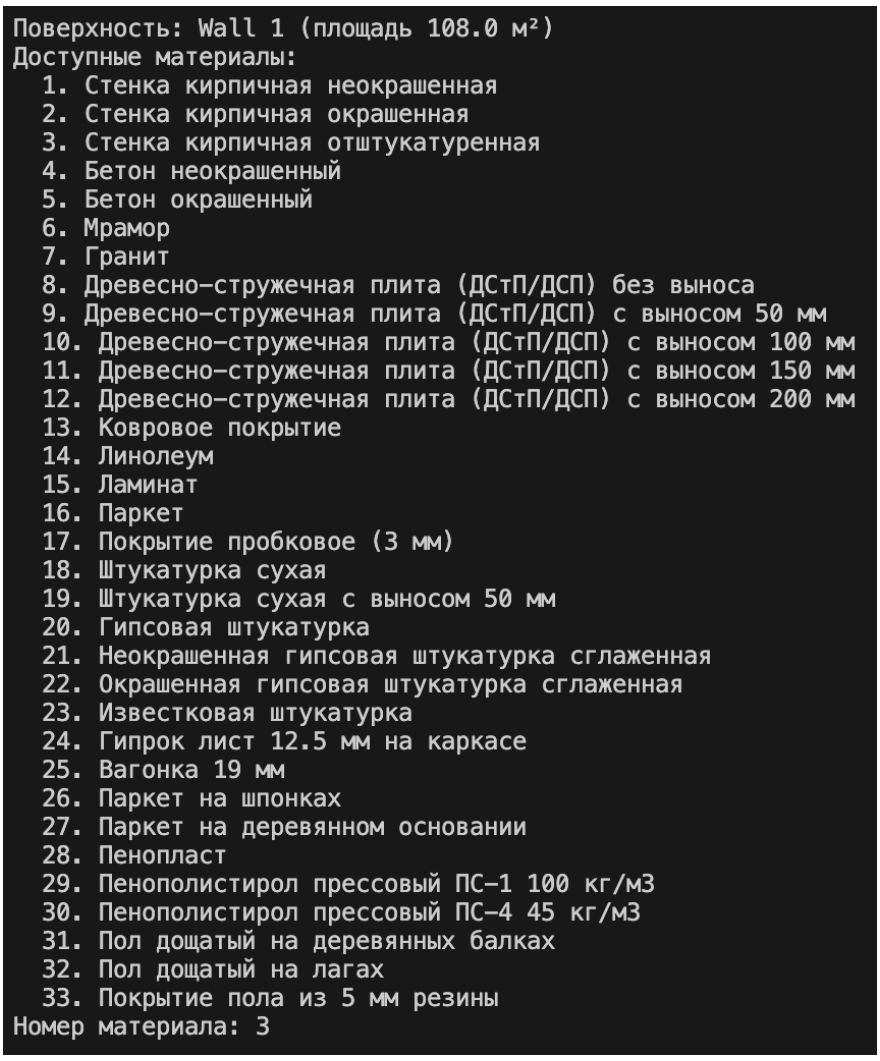


Рис. 14: Выбор материала поверхности

5.3.3 Ввод окон, дверей и элементов декора

После выбора материалов для всех поверхностей программа последовательно спрашивает:

- есть ли окна; если да — сколько, на какой стене и с какой площадью;
- есть ли двери; если да — сколько, на какой стене и с какой площадью;
- есть ли элементы интерьера (ковры, занавесы); если да — сколько, на какую поверхность, с каким материалом и с какой площадью.

При вводе площади программа контролирует оставшуюся свободную площадь поверхности и не даёт ввести значение, превышающее её.

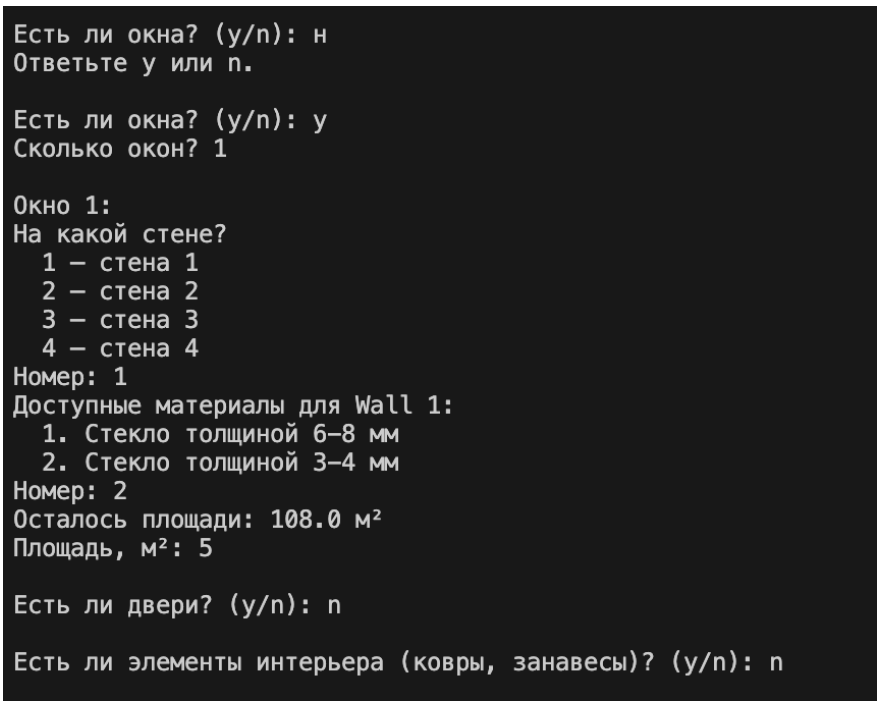


Рис. 15: Ввод элементов интерьера

5.3.4 Ввод сидячих мест

Далее программа предлагает указать количество типов сидячих мест (например, кресел или стульев). Для каждого типа мест выводится список материалов, из которого нужно выбрать подходящий, а затем — количество мест. Программа проверяет, чтобы суммарная площадь мест не превышала площадь пола.

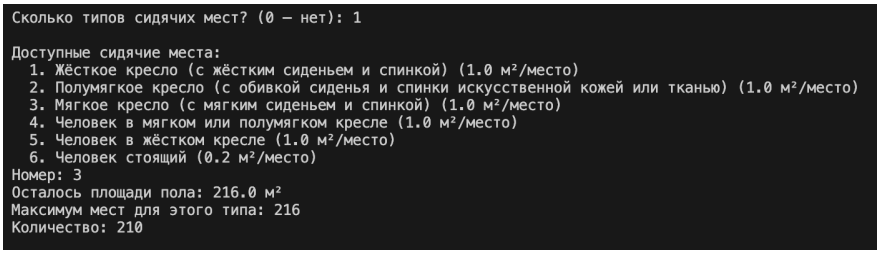


Рис. 16: Ввод сидячих мест

5.3.5 Проверка помещения и формирование рекомендаций

После ввода всех параметров программа предлагает выбрать тип назначения помещения из справочника, загружаемого из файла `data/purpose.csv`.

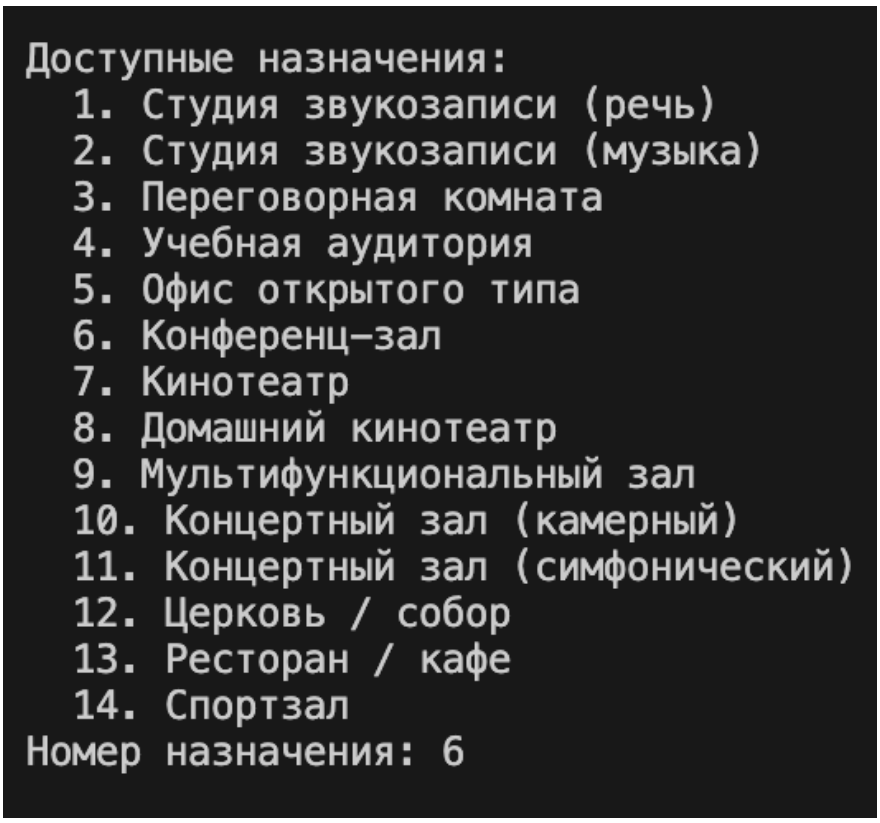


Рис. 17: Выбор типа назначения помещения

Затем выполняется акустический расчёт. Программа выводит результат в виде четырёх блоков:

- **Исходный результат** — рассчитанные значения эквивалентной площади звукопоглощения и времени реверберации по трём частотам (125, 500 и 2000 Гц);
- **Рекомендации** — список элементов, которые следует добавить или удалить;
- **Промежуточные результаты** — пошаговое изменение времени реверберации после каждого действия;
- **Итоговый результат** — состояние помещения после применения всех рекомендаций.

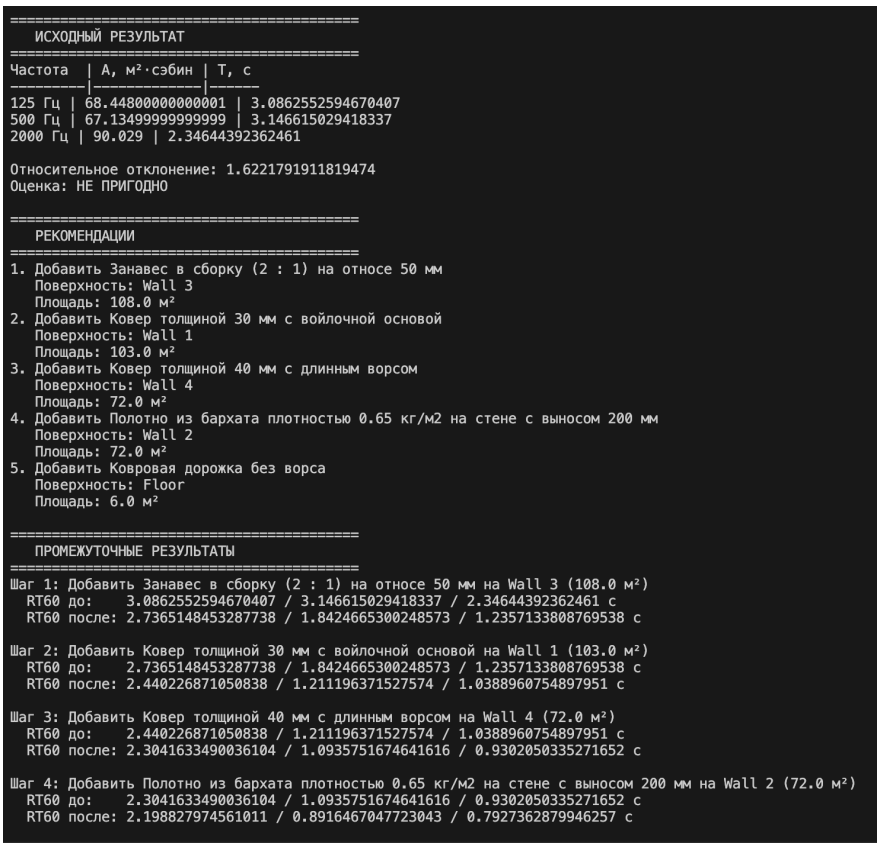


Рис. 18: Результат оценки и рекомендации

По завершении расчёта отчёт сохраняется в файл `data/reports/<имя помещения>.txt`, а событие фиксируется в журнале.

5.4 Создание и сохранение помещения

Для создания нового помещения с сохранением в JSON-файл необходимо выбрать пункт 2 главного меню. Порядок ввода параметров совпадает с пунктом 1, но после завершения ввода программа предлагает выбрать файл для сохранения:

```
1. Использовать существующий файл
2. Создать новый файл
0. Отмена
```

При выборе пункта 1 программа запрашивает имя существующего файла (без пути и без расширения `.json`); путь `data/rooms/` подставляется автоматически. Если файл не найден, программа предлагает ввести имя повторно.

При выборе пункта 2 программа запрашивает имя нового файла. Если файл с таким именем уже существует, программа запросит подтверждение перезаписи.

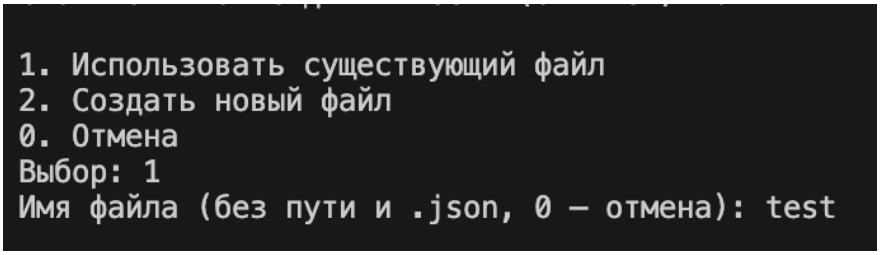


Рис. 19: Сохранение помещения в JSON-файл

5.5 Просмотр сохранённых помещений

Для просмотра списка сохранённых помещений необходимо выбрать пункт 4 главного меню. Программа выводит имя файла и список сохранённых в нём помещений.

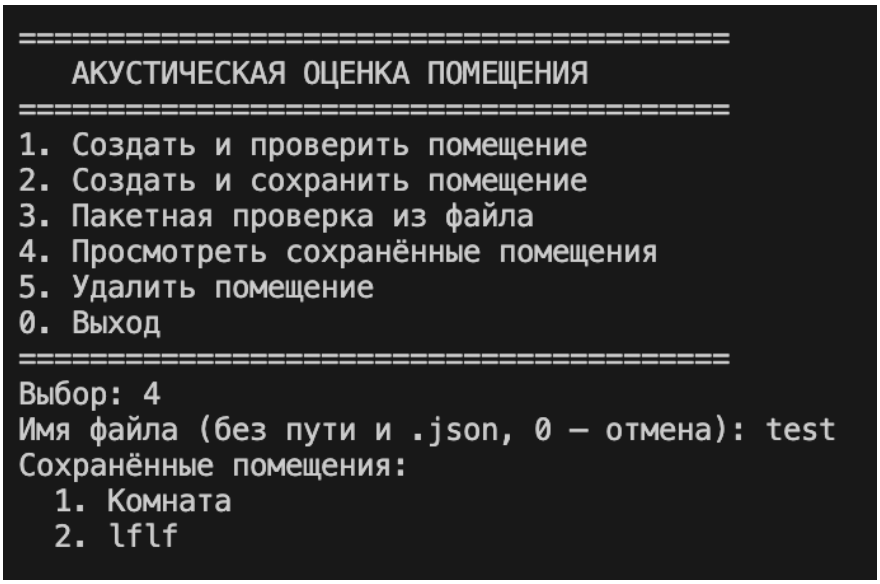


Рис. 20: Список сохранённых помещений

5.6 Удаление помещения

Для удаления помещения необходимо выбрать пункт 5 главного меню, указать файл и номер помещения в списке. Номер должен находиться в допустимом диапазоне — от 1 до количества помещений в файле. После удаления программа сохраняет обновлённый список.

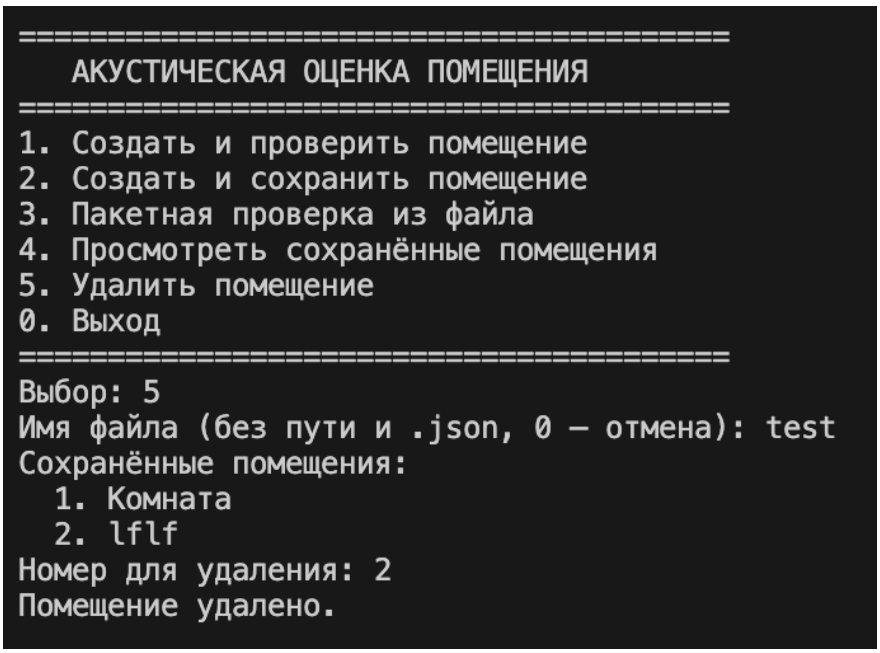


Рис. 21: Удаление помещения

5.7 Пакетная проверка помещений

Для проверки нескольких помещений из одного JSON-файла необходимо выбрать пункт 3 главного меню. Программа:

1. запрашивает имя файла с помещениями;
2. предлагает выбрать тип назначения помещения (единый для всего пакета);
3. для каждого помещения выполняет валидацию данных;
4. валидные помещения обрабатываются, невалидные — пропускаются с записью причины в журнал;
5. формирует сводный отчёт `data/summary.txt`.

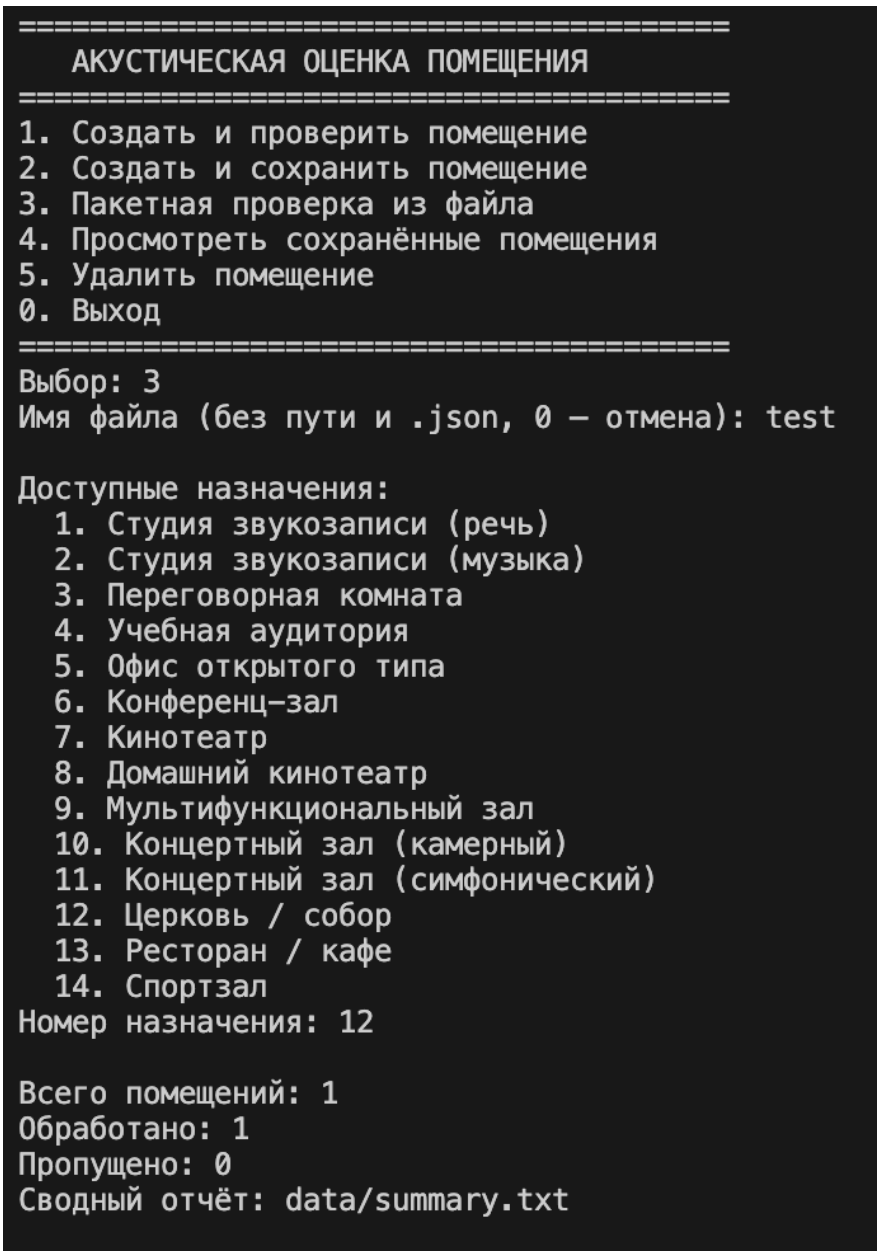


Рис. 22: Пакетная проверка помещений

Сводный отчёт содержит общую статистику (сколько помещений обработано, сколько пропущено), а также для каждого обработанного помещения — результат акустики до и после рекомендаций.

5.8 Обработка ошибок и журналирование

Программа обрабатывает некорректный пользовательский ввод без аварийного завершения. Если вместо числа введено нечисловое значение, отрицательное число или число вне допустимого диапазона, программа выводит сообщение об ошибке и повторно запрашивает ввод. При этом событие фиксируется в журнале с уровнем `Warning`.

Ошибки чтения и разбора CSV- и JSON-файлов, а также ошибки сохранения результатов обрабатываются через тип `Either AppError` и записываются в журнал с уровнем `Error`. Успешные операции, такие как сохранение помещения и формирование отчёта, фиксируются с уровнем `Info`.

Пример записи в журнал:

```
[2026-10-09 13:41:39] [INFO] === Запуск программы ===
[2026-10-09 13:44:18] [WARN] Помещение не пригодно: актовый зал
[2026-10-09 13:51:01] [INFO] Обработано: 1
[2026-10-09 13:51:01] [INFO] Обработано: Комната
[2026-10-09 13:59:15] [WARN] Некорректный пункт: 6
```

Рис. 23: Запись в лог

Журнал `data/app.log` содержит метки времени и уровни важности, что позволяет проследить все действия пользователя и события программы.

5.9 Завершение работы

Для корректного завершения работы программы необходимо выбрать пункт 0 главного меню. Программа фиксирует событие завершения в журнале и возвращает управление в командную строку.

Заключение

В результате выполнения курсового проекта разработана рекомендательная система акустической пригодности помещения по его назначению на языке Haskell с использованием системы сборки Stack.

В ходе работы были получены следующие результаты.

1. Разработана модульная архитектура приложения, включающая 14 модулей, организованных в иерархическую систему: модули типов данных предметной области, модули чтения CSV- и JSON-файлов, вычислительные модули (геометрия, поглощение, реверберация), модуль оценки пригодности, модуль формирования рекомендаций, модуль пакетной проверки, модуль формирования отчёта, модуль пользовательского ввода и модуль логирования.

2. Реализована система типов данных для типобезопасного представления основных сущностей предметной области: геометрических параметров помещения, материалов поверхностей, дверей, окон, элементов интерьера, сидячих мест, типов назначения, результатов акустических расчётов, оценки пригодности и рекомендаций. Для сериализации всех типов в формат JSON реализованы экземпляры классов `ToJSON` и `FromJSON`.

3. Реализованы алгоритмы акустической обработки данных в соответствии с математическим обеспечением технического задания:

   - расчёт объёма помещения, площадей поверхностей, площади зоны сидячих мест, общей и свободных площадей;
   - расчёт эквивалентной площади звукопоглощения с учётом вклада поверхностей, элементов интерьера, сидячих мест и добавочного поглощения по трём октавным частотам (125, 500 и 2000 Гц);
   - расчёт времени реверберации по формуле Сэбина или Эйринга с выбором формулы отдельно для каждой частоты в зависимости от среднего коэффициента звукопоглощения;
   - сравнение рассчитанного времени реверберации с нормативным

диапазоном выбранного назначения и оценка пригодности помещения;

- формирование последовательных рекомендаций по добавлению или удалению элементов интерьера с выбором пары «элемент — поверхность» по максимуму прироста эквивалентной площади поглощения на худшей частоте.

4. Реализована загрузка данных из CSV-файлов с использованием библиотеки `cassava` и сохранение помещений в формате JSON с использованием библиотеки `aeson`. Обработка ошибок ввода-вывода и разбора данных выполнена через тип `Either AppError`, что исключает аварийное завершение программы.

5. Разработан консольный интерфейс пользователя, предоставляющий доступ к следующим операциям: создание и немедленная проверка помещения, создание с сохранением в выбранный или новый JSON-файл, пакетная проверка нескольких помещений, просмотр сохранённых помещений, удаление помещения. Все вводимые значения проходят валидацию, некорректный ввод обрабатывается повторным запросом с записью предупреждения в журнал.

6. Реализованы формирование текстового отчёта по результатам расчёта и логирование действий пользователя и событий программы с уровнями `Info`, `Warning`, `Error`. Отчёт содержит результаты по трём частотам, оценку пригодности, рекомендации и пометку об оценочном характере расчёта.

7. Разработаны тесты свойств с использованием библиотеки **QuickCheck**. Тестами покрыты ключевые вычислительные функции: расчёт объёма, площади поверхностей, поглощения, времени реверберации по формулам Сэбина и Эйринга, отклонения от нормы и оценки пригодности. Все свойства успешно проходят проверку на 100 случайных наборах входных данных каждое.

8. Подготовлены диаграммы модулей и типов проекта, отражающие архитектуру приложения и связи между модулями.

История коммитов проекта

Ниже приведены коммиты, отражающие ход работы над проектом. Указаны короткий хэш и сообщение коммита.

- `b1c12ab` — first commit.
- `f568228` — Исправление цели, задач и исходных данных.
- `d10ef65` — Изменение ТЗ.
- `b348275` — Изменение темы и ТЗ.
- `653df98` — Обновление ТЗ.
- `9f6663a` — Изменение темы и ТЗ.
- `01f031d` — Исправление ТЗ.
- `ca4367d` — Добавление диаграммы, создание Stack, Types.
- `ef59d47` — Добавление модулей Calculation.
- `0de5e46` — Добавление модуля Evaluation.Suitability.
- `d418db0` — Добавление Input.CSV и подключение библиотеки cassava.
- `c4fa43a` — Изменение диаграммы, чтение и выгрузка JSON.
- `4a2c47c` — Добавление рекомендаций.
- `c42d9da` — Добавление Output.Report и изменение Acoustic.
- `fb0d2f7` — Add CLI, UPD CSV, UPD Main.
- `cef853c` — Add Logger, UPD CLI, UPD Диаграмма.
- `65f22c7` — Add test, UPD T.Material, UPD CSV, add csv-files, UPD Диаграмма.
- `613a5e0` — Исправление работы логов, исправление файла с материалами.

- `8e44146` — Правки ТЗ.
- `fc67a34` — Отчёт.
- `3f377cb` — Исправление титульного листа в отчёте.
- `ec4cea3` — Исправление цели ТЗ.
- `799901a` — Исправление ТЗ.
- `f972345` — Исправление диаграммы.
- `af11227` — Исправление некоторых типов и добавление нового.
- `6484bb9` — Исправление некоторых типов.
- `f73b349` — Добавление новых функций в Calculation.
- `4deead5` — Изменение функций и добавление новых у Acoustic, Suitability, Report.
- `3fcb8cb` — Исправление обработки CSV и JSON файлов.
- `b7868b3` — Исправление функций логов и добавление нового модуля Check.hs.
- `3a0721d` — Обновление CLI.
- `ce4398f` — Добавлены тесты.

Список литературы

1. ISO 3382-1:2009. Acoustics — Measurement of room acoustic parameters — Part 1: Performance spaces. — International Organization for Standardization, 2009.

2. Sabine W. C. Collected Papers on Acoustics. — Cambridge: Harvard University Press, 1922.

3. Pop-music: Что такое звукоизоляция и звукопоглощение? — URL: https://pop-music.ru/articles/zvukoizolyatsiya-i-zvukopogloshchenie/ (дата обращения: 25.08.2026).

4. aeson: Fast JSON parsing and encoding for Haskell // Hackage. — URL: https://hackage.haskell.org/package/aeson (дата обращения: 25.05.2026)

5. QuickCheck: Automatic testing of Haskell programs // Hackage. — URL: https://hackage.haskell.org/package/QuickCheck (дата обращения: 25.05.2026).

6. The Haskell Tool Stack. — URL: https://docs.haskellstack.org (дата обращения: 25.05.2026).